



UNIVERSIDADE FEDERAL DE SANTA CATARINA
CENTRO TECNOLÓGICO
DEPARTAMENTO DE ENGENHARIA DE AUTOMAÇÃO E SISTEMAS
CURSO DE GRADUAÇÃO EM ENGENHARIA DE CONTROLE E AUTOMAÇÃO

Aruã Viggiano Souza

Inteligência Artificial de Borda para Leitura Automática de Placas Veiculares

Florianópolis
2026

Aruã Viggiano Souza

Inteligência Artificial de Borda para Leitura Automática de Placas Veiculares

Relatório final da disciplina DAS5511 (Projeto de Fim de Curso) como Trabalho de Conclusão do Curso de Graduação em Engenharia de Controle e Automação da Universidade Federal de Santa Catarina em Florianópolis.

Orientador: Prof. Eric Aislan Antonelo, Dr.

Supervisor: Dennis Kerr Coelho

Florianópolis

2026

Ficha de identificação da obra

A ficha de identificação é elaborada pelo próprio autor.

Orientações em:

<http://portalbu.ufsc.br/ficha>

Aruã Viggiano Souza

Inteligência Artificial de Borda para Leitura Automática de Placas Veiculares

Esta monografia foi julgada no contexto da disciplina DAS5511 (Projeto de Fim de Curso) e aprovada em sua forma final pelo Curso de Graduação em Engenharia de Controle e Automação

Florianópolis, 26 de junho de 2026.

Prof. Marcelo De Lellis Costa de Oliveira, Dr.
Coordenador do Curso

Banca Examinadora:

[preencher somente após a defesa para a Versão Final da BU]

Prof(a). xxxx, Dr(a).
Orientador(a)
UFSC/CTC/EAS

xxxx, Eng(a).
Supervisor(a)
Empresa/Universidade xxxx

Prof(a). xxxx, Dr(a).
Avaliador(a)
Instituição xxxx

Prof. xxxx, Dr.
Presidente da Banca
UFSC/CTC/EAS

Este trabalho é dedicado aos meus colegas de classe e
aos meus queridos pais.

AGRADECIMENTOS

Inserir os agradecimentos aos colaboradores da execução do trabalho.

[illegible]

*“Texto da Epígrafe.
Citação relativa ao tema do trabalho.
É opcional. A epígrafe pode também aparecer
na abertura de cada seção ou capítulo.
Deve ser elaborada de acordo com a NBR 10520.”
(SOBRENOME do autor da epígrafe, ano)*

DECLARAÇÃO DE PUBLICIDADE

Florianópolis, 26 de junho de 2026.

Na condição de representante da Palmsoft Tecnologia na qual o presente trabalho foi realizado, declaro não haver ressalvas quanto ao aspecto de sigilo ou propriedade intelectual sobre as informações contidas neste documento, que impeçam a sua publicação por parte da Universidade Federal de Santa Catarina (UFSC) para acesso pelo público em geral, incluindo a sua disponibilização *online* no Repositório Institucional da Biblioteca Universitária da UFSC. Além disso, declaro ciência de que <o/a> autor<a>, na condição de estudante da UFSC, é obrigado a depositar este documento, por se tratar de um Trabalho de Conclusão de Curso, no referido Repositório Institucional, em atendimento à Resolução Normativa n° 126/2019/CUn.

Por estar de acordo com esses termos, subscrevo-me abaixo.

Dennis Kerr Coelho
Palmsoft Tecnologia

RESUMO

Instruções do padrão genérico de TCCs da BU: No Resumo são ressaltados o objetivo da pesquisa, o método utilizado, as discussões e os resultados com destaque apenas para os pontos principais. O resumo deve ser significativo, composto de uma sequência de frases concisas, afirmativas, e não de uma enumeração de tópicos. Não deve conter citações. Deve usar o verbo na voz ativa e na terceira pessoa do singular. O texto do resumo deve ser digitado, em um único bloco, sem espaço de parágrafo. O espaçamento entre linhas é simples e o tamanho da fonte é 12. Abaixo do resumo, informar as palavras-chave (palavras ou expressões significativas retiradas do texto) ou, termos retirados de thesaurus da área. Deve conter de 150 a 500 palavras. O resumo é elaborado de acordo com a NBR 6028.

Instruções da Coordenação de PFC: O Resumo deve descrever de forma sucinta: o contexto/motivação/problema tratado no PFC; a solução proposta; a implementação/desenvolvimento; a metodologia e as principais técnicas e ferramentas utilizadas; os principais resultados obtidos e a importância/impactos de tais resultados para a empresa/clientes da empresa/instituto de pesquisa. Escrever todos esses pontos de forma bem resumida e direta, e sem entrar em detalhes técnicos. O tamanho do Resumo deve ocupar praticamente esta página inteira, e num **único** parágrafo. Além disso, Resumo + Palavras-Chave não podem ultrapassar esta página.

Palavras-chave: Palavra-chave 1. Palavra-chave 2. Palavra-chave 3. *[essas palavras-chave devem obrigatoriamente ser utilizadas no Resumo]*

ABSTRACT

Resumo traduzido para outros idiomas, neste caso, inglês. Segue o formato do resumo feito na língua vernácula. As palavras-chave traduzidas, versão em língua estrangeira, são colocadas abaixo do texto precedidas pela expressão “Keywords”, separadas por ponto.

Keywords: Keyword 1. Keyword 2. Keyword 3.

LISTA DE FIGURAS

Figura 1 – Placa Radxa ROCK 3A	19
Figura 2 – Diagrama das etapas da leitura de placas (método 1)	21
Figura 3 – Diagrama das etapas da leitura de placas (método 2)	21
Figura 4 – Fases do processo CRISP-DM (Chapman et al., 2000)	28
Figura 5 – Camada convolucional (IBM Inc., 2024)	31
Figura 6 – Rede neural convolucional para classificação de imagens (Saha, 2018)	32
Figura 7 – Exemplo de detecção de objetos	33
Figura 8 – Diagrama de funcionamento do modelo YOLO (Redmon et al., 2016)	34
Figura 9 – Exemplo de detecção de pontos chave em um corpo humano	34
Figura 10 – Transformação de perspectiva (TheAILearner, 2023)	36
Figura 11 – Exemplo de separação de regiões por componentes conexos	38
Figura 12 – A NPU. (a) NPU de 8 <i>processing engines</i> ; (b) Fila de uma <i>processing engine</i> (Chen et al., 2020)	39
Figura 13 – Modelo de placa veicular do Brasil de 1990	50
Figura 14 – Modelo de placa veicular do Brasil de 1990 para veículos que realizam transporte pago	51
Figura 15 – Modelo de placa veicular do padrão Mercosul	51
Figura 16 – Exemplo de imagem do <i>dataset</i> com as marcações dos 4 cantos da placa	52
Figura 17 – Imagem em escala de cinza	55
Figura 18 – Extração da placa com base na <i>bounding box</i>	56
Figura 19 – Transformação de perspectiva com base nos quatro cantos da placa	57
Figura 20 – Medidas para remoção de bordas das placas do padrão antigo	57
Figura 21 – Medidas para remoção de bordas das placas do padrão Mercosul	58
Figura 22 – Remoção de bordas das placas	58
Figura 23 – Imagem de placa binarizada com o método de Sauvola	59
Figura 24 – Separação de caracteres para os dois modelos de placa	60
Figura 25 – Exemplo de limpeza de imagem de caractere com base em continuidade de objetos	60
Figura 26 – Diagrama da rede neural de classificação de letras	62
Figura 27 – Diagrama da rede neural de classificação de números	62
Figura 28 – Curvas de treinamento do modelo de detecção de placas	67
Figura 29 – Curvas de treinamento do modelo de classificação de números em placas do tipo Mercosul	74
Figura 30 – Curvas de treinamento do modelo de classificação de letras em placas do tipo Mercosul	75

Figura 31 – Curvas de treinamento do modelo de classificação de números em placas do tipo Brasil	75
Figura 32 – Curvas de treinamento do modelo de classificação de letras em placas do tipo Brasil	75
Figura 33 – Curvas de treinamento do modelo de classificação em placas do tipo Brasil	76
Figura 34 – Curvas de treinamento do modelo de classificação em placas do tipo Mercosul	77

LISTA DE QUADROS

LISTA DE TABELAS

Tabela 1 – Requisitos não funcionais de RF1	46
Tabela 2 – Requisitos não funcionais de RF2	47
Tabela 3 – Tamanhos do modelo de detecção de objetos YOLO26-pose	54

SUMÁRIO

1	INTRODUÇÃO	17
1.1	CONTEXTO DO TRABALHO	17
1.2	RECURSOS UTILIZADOS	18
1.2.1	Placa Radxa ROCK 3A	18
1.3	METODOLOGIA	19
1.3.1	Planejamento do modelo de leitura de placas	19
1.3.2	Planejamento da execução dos modelos na placa ROCK 3A	21
1.3.3	Planejamento das etapas do projeto	21
1.3.4	Preparação dos dados	22
1.3.5	Treinamento do modelo de detecção de placas	22
1.3.6	Desenvolvimento de algoritmo de pré-processamento e extração de caracteres	23
1.3.7	Geração de <i>datasets</i> de caracteres e placas	23
1.3.8	Treinamento de modelos de classificação de caracteres	24
1.3.9	Conversão dos modelos para formato compatível com NPU	25
1.3.10	Teste dos modelos no computador e na placa	25
2	FUNDAMENTAÇÃO TEÓRICA	26
2.1	TREINAMENTO DE MODELOS DE APRENDIZADO DE MÁQUINA	26
2.1.1	Etapas do padrão CRISP-DM	26
2.2	REDES NEURAIS E BACKPROPAGATION	28
2.3	REDES NEURAIS CONVOLUCIONAIS	30
2.4	MODELOS DE DETECÇÃO DE OBJETOS	32
2.4.1	Modelos YOLO	33
2.4.2	Detecção de pontos chave	34
2.5	TÉCNICAS DE TRANSFORMAÇÃO DE IMAGEM PARA VISÃO COMPUTACIONAL	35
2.5.1	Transformação de perspectiva	35
2.5.2	Limiarização adaptativa	36
2.5.3	Segmentação de caracteres	37
2.5.4	Filtragem de componentes conexos	37
2.6	NEURAL PROCESSING UNIT	38
2.7	QUANTIZAÇÃO DE REDES NEURAIS	39
3	DESCRIÇÃO DO PROBLEMA E REQUISITOS TÉCNICOS	41
3.1	DESCRIÇÃO DA EMPRESA	41
3.2	MOTIVAÇÃO	42
3.3	ANÁLISE DO PROBLEMA	43
3.3.1	Criação do algoritmo de leitura automática de placas de carros	43

3.3.2	Teste do algoritmo desenvolvido na placa Rock 3A	44
3.3.2.1	<i>Neural Processing Unit</i> da placa Rock 3A	45
3.4	REQUISITOS DA SOLUÇÃO	46
3.4.1	RF1: <i>Software</i> de Detecção Automática de Placas	46
3.4.1.1	Requisitos não funcionais:	46
3.4.2	RF2: Execução na Placa Rock 3A	46
3.4.2.1	Requisitos não funcionais:	47
4	SOLUÇÃO PROPOSTA E METODOLOGIA UTILIZADA	48
4.1	METODOLOGIA DE DESENVOLVIMENTO	48
4.2	ANÁLISE DOS DADOS	50
4.2.1	Características do objeto	50
4.2.2	Características do <i>dataset</i> utilizado	51
4.3	<i>PIPELINE</i> DE LEITURA DE PLACAS	52
4.3.1	Modelo de detecção de objetos e pontos-chave	53
4.3.2	Algoritmo de processamento da imagem	54
4.3.2.1	Conversão para escala de cinza	55
4.3.2.2	Extração da placa com base na <i>bounding box</i>	55
4.3.2.3	Transformação de perspectiva	56
4.3.2.4	Remoção das bordas	57
4.3.2.5	Binarização	58
4.3.2.6	Separação dos caracteres	59
4.3.2.7	Limpeza das imagens de caracteres	60
4.3.3	Modelos de classificação de caracteres	61
4.3.4	Modelos de extração e classificação de caracteres	62
4.4	CONVERSÃO PARA EXECUÇÃO NA PLACA ROCK 3A	63
4.5	METODOLOGIA DE TESTE	64
5	DESENVOLVIMENTO E ANÁLISE DOS RESULTADOS OBTIDOS	65
5.1	PREPARAÇÃO DOS DADOS	65
5.1.1	Organização dos dados	66
5.2	TREINAMENTO DO MODELO DE DETECÇÃO DE PLACAS	66
5.2.1	Exportação do modelo YOLO	68
5.3	IMPLEMENTAÇÃO DO ALGORITMO DE EXTRAÇÃO DE CARACTERES	68
5.3.1	Inferência com o modelo de detecção de placas	68
5.3.2	Criação de nova imagem contendo apenas a placa	70
5.3.3	Correção de perspectiva	70
5.3.4	Remoção das bordas das placas e binarização	71
5.3.5	Separação dos caracteres	71
5.3.6	Limpeza das imagens de caracteres	72

5.4	CRIAÇÃO DOS <i>DATASETS</i> DE CARACTERES E DE PLACAS . . .	73
5.5	TREINAMENTO DOS MODELOS DE CLASSIFICAÇÃO DE CARAC- TERES	73
5.5.1	Classificação de caracteres isolados	74
5.5.2	Extração e classificação simultânea	75
5.6	QUANTIZAÇÃO E TRANSFORMAÇÃO DOS MODELOS PARA FOR- MATO COMPATÍVEL COM O <i>HARDWARE</i>	77
5.6.1	Resultados da conversão	78
5.6.2	Criação dos datasets de quantização	78
5.7	PREPARAÇÃO DO <i>HARDWARE</i>	78
5.8	TESTES REALIZADOS	78
6	CONCLUSÃO	79
	REFERÊNCIAS	80
	APÊNDICE A – DESCRIÇÃO 1	82
	ANEXO A – DESCRIÇÃO 2	83

1 INTRODUÇÃO

Nos últimos anos a leitura automática de placas de veículos se tornou uma aplicação presente no cotidiano de muitos. Inúmeros estacionamentos de shoppings, supermercados e outros estabelecimentos de grande ou médio porte descobriram nessa tecnologia uma forma de melhorar a experiência do cliente que utiliza o estacionamento. O conceito é simples: quando o carro se aproxima da entrada a placa é imediatamente detectada e associada ao *ticket*. Então, quando o cliente se aproxima da cancela de saída, o sistema lê a placa novamente e já consegue verificar qual *ticket* está associado àquele placa e se ele já foi pago ou está na tolerância. Se o *ticket* está liberado a cancela abre e deixa o carro passar sem que o motorista precise aproximar o *ticket* do leitor.

Esse sistema gera as seguintes vantagens para os motoristas e para o próprio estabelecimento:

- Agiliza a saída dos carros, evitando formação de filas
- Permite que *tickets* perdidos possam ser recuperados digitalmente
- Cria a possibilidade de o estabelecimento cadastrar certas placas para receber acesso automático, sem precisar de *ticket*, facilitando o acesso dos funcionários.

Esses benefícios facilitam a vida dos usuários do estacionamento, melhorando a experiência daqueles que frequentam o estabelecimento e consequentemente agregando valor ao negócio.

Essa é apenas uma das possíveis aplicações da leitura automática de placas de veículo, que pode ser utilizada também em aplicações de segurança, auditoria, controle de tráfego e controle de acesso.

Nesse contexto, o desenvolvimento de modelos eficazes, rápidos e leves para leitura automática de placas é de extremo interesse econômico, pois pode permitir a aplicação desses modelos em dispositivos de baixo custo e aumentar sua adoção em diversas aplicações. Isso é o que esse trabalho se propõe a realizar.

1.1 CONTEXTO DO TRABALHO

O interesse nesse trabalho surgiu no contexto de um condomínio autônomo focado em aluguel para estadias de curto prazo. Nesse condomínio os hóspedes reservam os apartamentos em sites como *Booking* ou *AirBnB*. Os administradores do apartamento então cadastram a reserva em um sistema que libera automaticamente a entrada do hóspede no condomínio durante o período da reserva.

O acesso do hóspede ao condomínio se dá por reconhecimento facial, mas no estacionamento é utilizada uma câmera com leitura de placas de veículo integrada.

Só de aproximar seu carro, o portão já se abre, pois a placa é reconhecida como pertencente à um hóspede com reserva ativa e quando a estadia chega ao fim, o acesso é imediatamente revogado.

A câmera que torna isso possível é um dispositivo da Intelbras, modelo VIP 5460 LPR IA, cujo valor de mercado varia entre R\$ 5.700 e R\$ 7.500, longe de um preço acessível para aplicações de menor escala ou casos em que a quantidade de dispositivos necessários é maior.

Esse trabalho, então, se propõe a realizar uma prova de conceito: Criar um modelo de leitura automática de placas e executá-lo em uma placa de baixo custo. Esse seria o primeiro passo para o desenvolvimento de um protótipo barato de câmera com leitura automática de placas.

1.2 RECURSOS UTILIZADOS

Os recursos disponíveis para a realização desse projeto foram:

- Computador com CPU Intel Core i7 e GPU NVIDIA GeForce RTX 3060
- Placa Radxa ROCK 3A com Rockchip RK3568
- Micro SD 32 GB

1.2.1 Placa Radxa ROCK 3A

A placa ROCK 3A da Radxa foi o hardware disponibilizado para executar o modelo de leitura automática de placas a ser desenvolvido. Essa placa é equipada com um chip Rockchip RK3568, o qual possui os seguintes componentes:

- CPU Quad-core Arm® Cortex™-A55, 64-bit, 2GHz
- GPU Arm® Mali™ G52
- NPU 0.8 TOPs, INT8

A figura 1 mostra a vista superior da placa ROCK 3A.

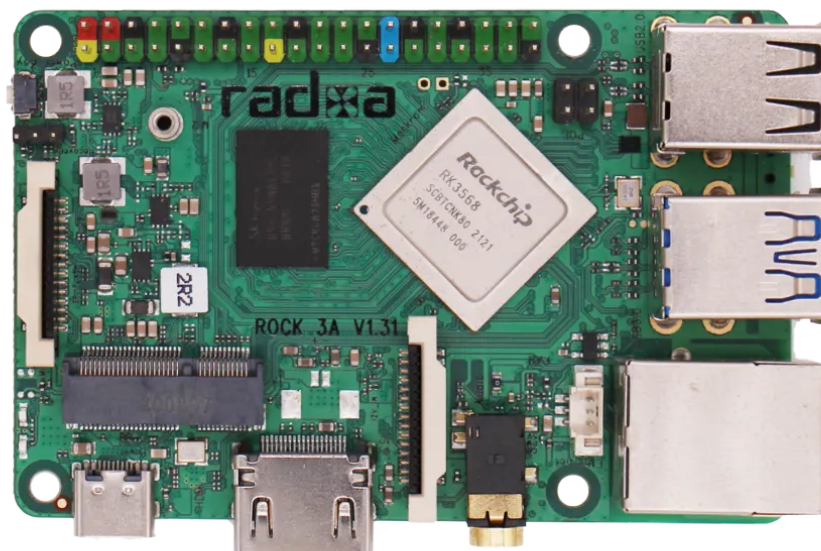


Figura 1 – Placa Radxa ROCK 3A

1.3 METODOLOGIA

A metodologia para o desenvolvimento do projeto começou com o planejamento da solução. Para isso, foi necessário quebrar o problema em problemas menores que podem ser resolvidos mais facilmente.

A primeira quebra a ser feita é a separação do problema de leitura de placas de carro do problema do hardware que executará a solução. Primeiro é necessário atingir um resultado satisfatório em ambiente de desenvolvimento para só então evoluir para o ambiente final da prova de conceito.

1.3.1 Planejamento do modelo de leitura de placas

O *input* com o qual a solução irá lidar é uma imagem na qual aparece a placa de um carro. Porém, apenas a placa em si nos interessa, sendo todo o resto da imagem informação desnecessária para o nosso objetivo. Portanto, o primeiro problema consiste em transformar a imagem que contém a placa de um carro em uma imagem que contém *somente* a placa de um carro. Todo o resto deve ser removido.

A partir da imagem da placa do carro, o próximo desafio é identificar onde estão as letras e os números. Modelos de classificação de imagens, principalmente de

imagens de caracteres, são extremamente eficazes, mas precisam que a imagem contenha única e exclusivamente o caractere a ser classificado. Dessa forma, a imagem contendo a placa inteira precisa ser quebrada em pedaços, cada um contendo um caractere.

Para que essa divisão da placa em caracteres tenha sucesso, é necessário saber como os caracteres estão dispostos pela placa, o que varia conforme o tipo da placa. No Brasil existem dois de *layout* de placa permitidos: O tipo antigo, ainda em circulação, e o tipo Mercosul, que passou a ser obrigatório para veículos novos no Brasil em 2020.

Por fim, o último problema é classificar cada imagem de caractere. Tendo as imagens classificadas, e sendo a sua ordem na placa conhecida, é possível inferir a placa do veículo. Com isso, o problema do modelo estaria resolvido.

A conclusão é que o problema do modelo de leitura de placas de carro pode ser dividido nos seguintes três problemas:

- Detecção e classificação da placa
- Extração dos caracteres
- Classificação dos caracteres

A detecção e classificação são uma única etapa pois modelos de detecção de objetos costumam ter a capacidade de detectar objetos de múltiplas classes e identificar qual é a classe de cada objeto detectado. Isso permite que a detecção da placa e a classificação do tipo de placa sejam feitos de uma só vez.

Para que a extração e classificação dos caracteres funcione bem, é vital que haja um pré-processamento da imagem da placa extraída pela detecção, com objetivo de remover ao máximo o ruído da imagem e deixando-a o mais limpa possível.

Uma outra possibilidade é tratar o problema de extração e classificação dos caracteres como um único problema a ser resolvido por um modelo mais robusto de inteligência artificial que realiza as duas tarefas de uma só vez. Essa alternativa é mais simples mas pode gerar bons resultados se a quantidade de dados disponível para ajuste do modelo for suficiente.

Os diagramas das figuras 2 e 3 demonstram as etapas do algoritmo em ambas as metodologias. O método 1 separa a extração e classificação dos caracteres e o método 2 os executa juntos.

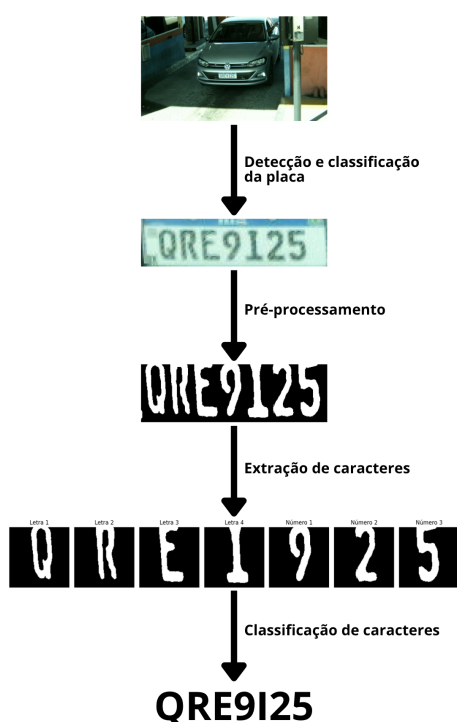


Figura 2 – Diagrama das etapas da leitura de placas (método 1)

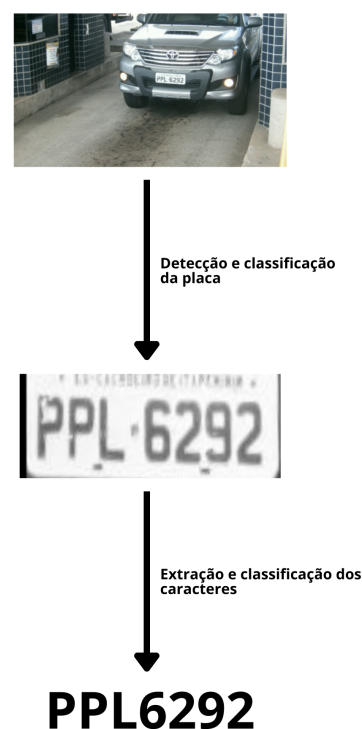


Figura 3 – Diagrama das etapas da leitura de placas (método 2)

1.3.2 Planejamento da execução dos modelos na placa ROCK 3A

A placa ROCK 3A é equipada com uma NPU (*Neural Processing Unit*), que em tese executa modelos de redes neurais de forma mais eficiente que uma CPU. Portanto, nosso objetivo é que os modelos sejam executados na NPU.

Para isso são necessárias duas etapas principais: Preparação da placa e preparação dos modelos.

A preparação da placa envolve:

- Instalar o sistema operacional
- Instalar todo o software necessário para inferência
- Ativar a NPU

A preparação dos modelos consiste basicamente em convertê-los para o formato RKNN, que é o formato aceito para a NPU do ROCK 3A.

1.3.3 Planejamento das etapas do projeto

Tendo o planejamento do modelo e da execução na placa, podemos agora pensar nas etapas do projeto. Essas etapas devem abarcar o desenvolvimento do modelo, a execução no *hardware* do projeto e o teste final dos resultados.

As etapas sequenciais, portanto são:

1. Preparação dos dados a serem utilizados para treinamento e teste do modelo
2. Treinamento de um modelo de detecção de placas (apenas identifica o tipo e a posição da placa)
3. Desenvolvimento de um algoritmo de pré-processamento e extração de caracteres
4. Geração de um *dataset* de caracteres extraídos e um *dataset* de placas extraídas
5. Treinamento de modelos de classificação de caracteres
6. Conversão dos modelos para formato compatível com NPU
7. Teste dos modelos no computador e na placa

1.3.4 Preparação dos dados

A preparação inicial dos dados envolveu principalmente dois procedimentos: Separação e organização.

A separação consistiu em separar os dados em três subgrupos, sendo eles **treino, validação e teste**. Os subgrupos possuem a função de impedir que dados utilizados no desenvolvimento da solução sejam também utilizados para avaliar a qualidade da mesma, de forma a não gerar um viés nos testes.

A organização, por outro lado, foi feita tendo em mente o algoritmo de treinamento do modelo de detecção de placas, que requer que os dados estejam organizados em uma dada estrutura, como veremos mais à frente. Além disso, o *dataset* de teste foi organizado para os testes da solução completa, a serem realizados no final do projeto.

1.3.5 Treinamento do modelo de detecção de placas

O modelo de detecção de placas possui três funções:

- Detectar os limites da placa na imagem (superior, inferior, esquerdo e direito)
- Detectar as coordenadas dos quatro cantos da placa
- Detectar a classe da placa entre duas possibilidades: Brasil e Mercosul

Sendo que a classe "Brasil" refere-se ao modelo antigo de placas brasileiras, enquanto "Mercosul" refere-se ao modelo novo, adotado em todo o bloco Mercosul.

Para essa funcionalidade o método utilizado foi treinar um modelo pré-treinado de detecção de objetos, adaptando-o para a necessidade específica do projeto. A

biblioteca Ultralytics, do Python, oferece uma interface para esse tipo de treinamento de modelos de detecção de objetos. Essa interface requer uma organização específica dos dados, que foi realizada na etapa anterior.

1.3.6 Desenvolvimento de algoritmo de pré-processamento e extração de caracteres

O modelo de detecção de placas de carro entrega a posição da placa na imagem e as coordenadas dos quatro cantos. A partir dessa informação é trivial cortar apenas a parte da imagem que contém a placa. Isso já ajuda bastante no processo de extração dos caracteres, mas não é suficiente. Uma questão importante é que em muitos casos a placa possui certa rotação ou perspectiva, o que pode atrapalhar bastante. Para corrigir isso é possível aplicar uma transformação linear com base nas coordenadas dos quatro cantos que corrige essa distorção.

Além disso, as placas possuem partes que não nos interessam. Apenas os caracteres são relevantes para a leitura, sendo todo o resto (parte superior, inferior, laterais) descartável.

Um outro fato importante é que é possível saber previamente quais caracteres são letras e quais são números com base em sua posição na placa. Tanto a placa Brasil quanto a Mercosul possui uma sequência bem definida de letras e números, de forma que basta saber qual é a classe da placa para que se saiba onde as letras e os números estão posicionados.

Com base nesses fatos, o algoritmo de pré-processamento e extração de caracteres deve:

- Corrigir a perspectiva e rotação da placa para que ela fique reta e com dimensões padronizadas
- Cortar os cantos que não possuem informações relevantes
- Fazer uma limpeza e remover todo o ruído desnecessário
- Dividir a placa em pedaços, cada um contendo um caractere
- Padronizar as dimensões das imagens de caracteres
- Separar os caracteres extraídos em letras e números com base na posição

1.3.7 Geração de *datasets* de caracteres e placas

Para que seja possível treinar os modelos de classificação dos caracteres é necessário criar *datasets* correspondentes aos modelos que se deseja ajustar. No caso dos modelos que realizam classificação de caracteres já extraídos é necessário

criar um *dataset* de imagens de caracteres. Para isso, tendo sido realizadas as etapas anteriores, basta aplicar a detecção de placa e extração de caracteres em cada imagem dos dados de treinamento. Como sabemos previamente quais placas são do tipo Brasil e quais são Mercosul, assim como também sabemos quais caracteres são números e quais são letras, é possível criar *datasets* separados para cada categoria, resultando em quatro *datasets*:

- Números Brasil
- Letras Brasil
- Números Mercosul
- Letras Mercosul

Já no caso dos modelos de classificação e extração de caracteres, o *dataset* de treinamento não pode conter caracteres extraídos, e sim placas de carro extraídas das imagens originais a partir do *output* do modelo de detecção de placas. Para esse modelo, muito do processamento realizado nas imagens de placas não é necessário, mas a correção de perspectiva ainda é interessante de ser utilizada para corrigir distorções e padronizar as dimensões das imagens de placas.

O *dataset* de placas, portanto, é formado por imagens de placas com dimensões fixas, que podem ainda ser divididos em:

- Placas Brasil
- Placas Mercosul

1.3.8 Treinamento de modelos de classificação de caracteres

O treinamento dos modelos de classificação de caracteres é dividido em duas partes, uma para a criação dos modelos de classificação de caracteres extraídos e outra para os modelos de extração e classificação. Os modelos de caracteres extraídos utilizam o *dataset* de imagens de caracteres para seu ajuste, enquanto os modelos de extração e classificação utilizam os *datasets* de placas.

A tarefa de classificar caracteres isolados não é particularmente complexa, portanto uma rede neural convolucional simples já é suficiente para esse problema. Como os dados de caracteres estão separados por modelo de placa e por tipo (letra/número) foram criados quatro modelos diferentes, um para cada combinação. Todos possuem a mesma arquitetura, exceto pela camada de saída, que possui 26 classes nos modelos de letras e 10 classes nos modelos de números.

Os modelos de extração e classificação, por outro lado, deve ser muito mais complexos, pois são incumbidos de uma tarefa muito mais difícil. Esses modelos devem

receber na entrada uma imagem de uma placa de carro e convertê-la em 7 caracteres, cada um sendo representado por um vetor de 26 ou 10 valores, dependendo se aquele caractere é uma letra ou um número.

Como existem dois modelos de placa e cada um possui um padrão de letras e números, foram criados dois modelos dessa modalidade, um para cada padrão de placa.

1.3.9 Conversão dos modelos para formato compatível com NPU

O formato compatível com a NPU do ROCK 3A é o formato RKNN. A própria documentação da placa indica uma biblioteca chamada *rknn toolkit* que oferece a *API* tanto para conversão dos modelos para esse formato quanto para inferência com esses modelos. Porém, a conversão não é tão trivial, pois precisa ser feita em um ambiente *linux* de uma determinada distribuição e versão. Além disso, os modelos precisam ser exportados para o formato ONNX antes de serem convertidos, pois o *rknn toolkit* faz a conversão entre esses dois formatos.

Além desses requisitos obrigatórios para a conversão, existe um requisito opcional, porém recomendado. Este é que durante a conversão para RKNN os modelos sejam quantizados para *uint8*, ou seja, que seus pesos sejam convertidos de números reais do tipo *float* de 32 *bits* para números inteiros positivos de 8 *bits*. Isso é recomendado pois intensifica a performance da NPU.

Para a quantização é necessário criar um *dataset* de quantização. Esse *dataset* só precisa conter dados de entrada, não sendo necessários os rótulos.

1.3.10 Teste dos modelos no computador e na placa

Uma vez que os modelos estão prontos e convertidos é possível testá-los tanto no computador quanto na placa. No computador ainda é possível testá-los na CPU e na GPU. Para avaliar a performance nos diferentes cenários, vamos usar o *dataset* de teste que foi gerado no começo e avaliar em quantas imagens o modelo acerta a placa do carro.

Porém, a principal diferença entre os locais de execução não deve vir no percentual de acerto, e sim no tempo de inferência. Portanto, é necessário medir o tempo, tanto do modelo de detecção de placa quanto nos modelos de classificação de caracteres.

2 FUNDAMENTAÇÃO TEÓRICA

Neste capítulo é discutida a fundamentação teórica dos principais conceitos, técnicas e tecnologias utilizados no projeto, incluindo:

- Treinamento de modelos de aprendizado de máquina
- Redes neurais e *backpropagation*
- Redes neurais convolucionais
- Modelos de detecção de objetos
- Técnicas de transformação de imagens para visão computacional
- *Neural Processing Unit*
- Quantização de redes neurais

Essa fundamentação forma a base necessária para a criação, desenvolvimento e validação do algoritmo de leitura de placas de carro desenvolvido nesse projeto, além de sua execução no *hardware* específico disponível.

2.1 TREINAMENTO DE MODELOS DE APRENDIZADO DE MÁQUINA

O processo de treinamento de modelos de aprendizado de máquina nada mais é do que uma variação específica do processo padrão de mineração de dados, que consiste no ato de utilizar algoritmos para extrair informações relevantes de um banco de dados (Fayyad; Piatetsky-Shapiro; Smyth, 1996).

Este processo já possui uma forma bem definida e aceita conhecida como CRISP-DM (*Cross-Industry Standard Process for Data Mining*) (Chapman et al., 2000).

O treinamento de modelos de aprendizado de máquina se enquadra como um processo de mineração de dados, pois nada mais é do que encontrar um modelo que, de alguma forma, represente o relacionamento entre diferentes aspectos do conjunto de dados. O modelo, nesse caso, representa a informação que se deseja extrair dos dados.

Um bom modelo descreverá com precisão o relacionamento entre os aspectos desejados do conjunto de dados e será portanto útil para extrair novas informações de novos dados.

2.1.1 Etapas do padrão CRISP-DM

O padrão CRISP-DM nada mais é que uma sequência de etapas a partir das quais é possível extrair informações úteis de um banco de dados, sendo que cada etapa é definida por tarefas e um resultado esperado.

As etapas são:

1. **Entendimento do problema**

A primeira fase consiste em entender as características do problema e os requisitos da solução, criando assim uma definição de problema de mineração de dados e um plano preliminar para atingir os objetivos (Chapman et al., 2000).

2. **Entendimento dos dados**

A fase seguinte envolve se familiarizar com os dados à disposição. Isso envolve descobrir *insights* iniciais nos dados, detectar problemas de qualidade dos dados e identificar subsets interessantes (Chapman et al., 2000).

3. **Preparação dos dados**

Essa etapa consiste nas atividades necessárias para construir o *dataset* final que será utilizado a partir dos dados brutos. Pode envolver diversas tarefas como limpeza dos dados, seleção de atributos relevantes e transformação dos dados (Chapman et al., 2000).

4. **Modelagem**

A modelagem envolve selecionar, aplicar e calibrar técnicas diversas de modelagem, buscando encontrar um modelo ótimo para o problema que se deseja resolver (Chapman et al., 2000).

5. **Avaliação**

Após chegar em um ou mais modelos que aparentam ser apropriados, uma avaliação final deles deve ser feita para concluir se atendem aos requisitos de negócio. Ao final dessa avaliação deve-se chegar em uma conclusão relativa ao uso ou não de cada modelo para o objetivo final (Chapman et al., 2000).

6. **Deployment**

Por fim, a última etapa é o momento em que o conhecimento obtido é preparado para uso pelo cliente do projeto. Essa preparação depende muito dos requisitos, podendo ser em forma de um relatório ou de um sistema automatizado de mineração de dados (Chapman et al., 2000).

A figura 4 mostra um diagrama do processo e demonstra como algumas das etapas possuem uma relação de troca, de forma que o conhecimento ou os desafios encontrados em uma dada etapa podem fazer com que seja necessário voltar para uma etapa anterior.

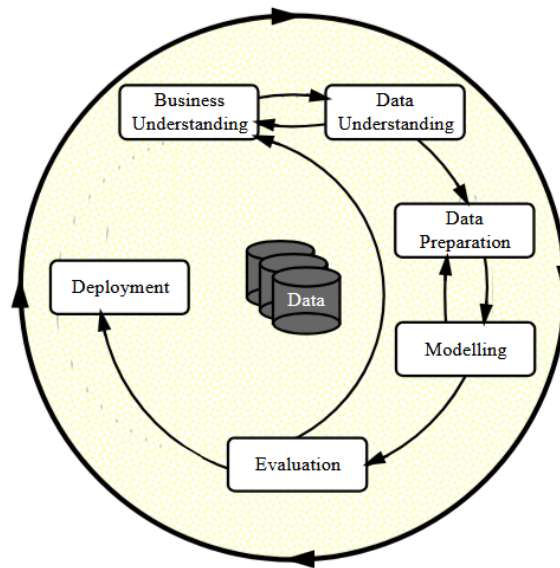


Figura 4 – Fases do processo CRISP-DM (Chapman et al., 2000)

2.2 REDES NEURAIS E BACKPROPAGATION

As redes neurais artificiais e o algoritmo de *backpropagation* são alguns dos fundamentos mais importantes da inteligência artificial moderna. As redes neurais permitem a criação de modelos capazes de aproximar relações não lineares e complexas entre dados, sendo que o limite da complexidade dessas relações se dá apenas pela limitação computacional ou do volume de dados utilizados no ajuste do modelo. Já o algoritmo de *backpropagation* permite ajustar o modelo de redes neurais aos dados, minimizando uma função de erro.

Esses conceitos são utilizados para criar modelos de inteligência artificial voltados para visão computacional, processamento de linguagem natural, séries temporais, IA generativa e inúmeras outras aplicações.

O tipo clássico de redes neurais é composto por uma série de camadas de processamento. Cada camada processa a saída da camada anterior em sequência até chegar no *output* final. A saída $a_{(c)}$ de uma camada c é definida por:

$$a_{(c)} = f(z_{(c)}) \quad (1)$$

Sendo que f é a função de ativação da camada e $z_{(c)}$ é definido como:

$$z_{(c)} = W_{(c)} \times a_{(c-1)} + b_{(c)} \quad (2)$$

Sendo:

- $W_{(c)}$ a matriz de pesos da camada
- $a_{(c-1)}$ a saída da camada anterior

- $b_{(c)}$ o vetor de viés da camada

A função de ativação f é responsável por conferir não-linearidade à camada, pois sem ela a camada não passaria de uma transformação linear capaz apenas de representar relações lineares entre os dados, limitando muito sua aplicabilidade.

Existem diversas funções de ativação que podem ser usadas, mas as mais comuns são:

- **Sigmoide:** $\sigma(x) = \frac{1}{1+e^{-x}}$
- **Tangente hiperbólica:** $\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$
- **ReLU (*Rectified Linear Unit*):** $\text{ReLU}(x) = \begin{cases} x ; & x > 0 \\ 0 ; & x \leq 0 \end{cases}$

A matriz de pesos $W_{(c)}$ e o vetor de viés $b_{(c)}$ são compostos por valores ajustáveis que devem ser modificados até que o modelo esteja satisfatório.

Para que os pesos do modelo sejam ajustados é utilizado o algoritmo de *back-propagation* para calcular o gradiente da função de erro em relação aos pesos da rede. A função de erro varia conforme o objetivo daquela rede neural e da função de ativação da última camada.

Para calcular o gradiente, então, é necessário calcular o delta $\delta_{(c)}$ de cada camada.

Para a camada de saída da rede (C) o delta é dado por:

$$\delta_{(C)} = \hat{y} - y \quad (3)$$

Sendo que $\hat{y}$ é o valor predito e y é o valor esperado (verdadeiro).

Para as demais camadas o delta é definido como:

$$\delta_{(c)} = W_{(c+1)}^T \times \delta_{c+1} \odot f'(z_{(c)}) \quad (4)$$

E por fim, o gradiente da função de erro L em relação aos pesos fica:

$$\frac{\partial L}{\partial W_{(c)}} = \delta_{(c)} \times a_{(c-1)}^T \quad (5)$$

$$\frac{\partial L}{\partial b_{(c)}} = \delta_{(c)} \quad (6)$$

Nota-se que, enquanto a saída de uma camada é calculada com base na saída da camada anterior, o gradiente de uma camada é calculado com base no delta da camada seguinte. Essa propagação retroativa dos erros é o motivo de o algoritmo de cálculo do gradiente ser chamado de *backpropagation* (Rumelhart; Hinton, Geoffrey E.; Williams, 1986).

O último fator importante a ser discutido no que diz respeito às redes neurais artificiais é o algoritmo de atualização dos pesos com base no gradiente. Existem diversos algoritmos diferentes que podem ser utilizados, porém o mais clássico e simples é o SGD (*stochastic Gradient Descent*), no qual a um peso $v_{(c)}$ é atribuído:

$$v_{(c)} \leftarrow v_{(c)} - \eta \frac{\partial L}{\partial v_{(c)}} \quad (7)$$

Onde η é a taxa de aprendizagem.

2.3 REDES NEURAIS CONVOLUCIONAIS

A eficácia de redes neurais convolucionais (CNNs) para tarefas de classificação e segmentação de imagens já é comprovada (Krizhevsky; Sutskever; Hinton, Geoffrey E, 2012), sendo uma solução amplamente utilizada em diversas aplicações.

Essa modalidade de rede neural possui como característica as camadas convolucionais, que possui vantagens em relação as camadas *fully connected* tradicionais para tarefas de interpretação de imagens ou mesmo de sinais (LeCun et al., 1998).

A limitação das camadas *fully connected* para esse tipo de tarefa se dá pelo fato de que esse tipo de camada produz sua saída com base no relacionamento entre todos os *inputs* da camada uns com os outros. Isso é ótimo para achar relações entre diferentes atributos quando se trabalha com dados tabulares, por exemplo, mas não é tão bom para encontrar características específicas que podem estar localizadas em diferentes partes de uma imagem.

Para esse tipo de tarefa as camadas convolucionais são mais apropriadas, pois utilizam conectividade local e compartilhamento de pesos, reduzindo drasticamente a quantidade de parâmetros necessários para se ter um resultado satisfatório.

A camada convolucional funciona deslizando pelo *input* um filtro chamado *kernel*, que nada mais é do que uma matriz de pesos de dimensão pequena, como 3x3 ou 5x5. Em cada posição do *kernel* é então feita uma transformação linear de um conjunto local de valores utilizando os pesos do *kernel*.

Fazendo isso em todo o *input* obtêm-se um mapa de características, que destaca padrões identificados em cada parte do *input*. Esse mapa de características pode então ser alimentado para outra camada convolucional e assim por diante. Além disso, para cada *input* pode-se (e deve-se) ter múltiplos *kernels*, de forma a gerar múltiplos mapas de características em cada camada, cada um destacando aspectos distintos do *input*. A imagem 5 exemplifica o funcionamento da camada convolucional.

A equação da convolução 2D discreta utilizada nas camadas convolucionais para imagens é:

$$(I * K)(x, y) = \sum_i \sum_j I(x + i, y + j) \cdot K(i, j) \quad (8)$$

Sendo que I é a imagem e K é o *kernel*.

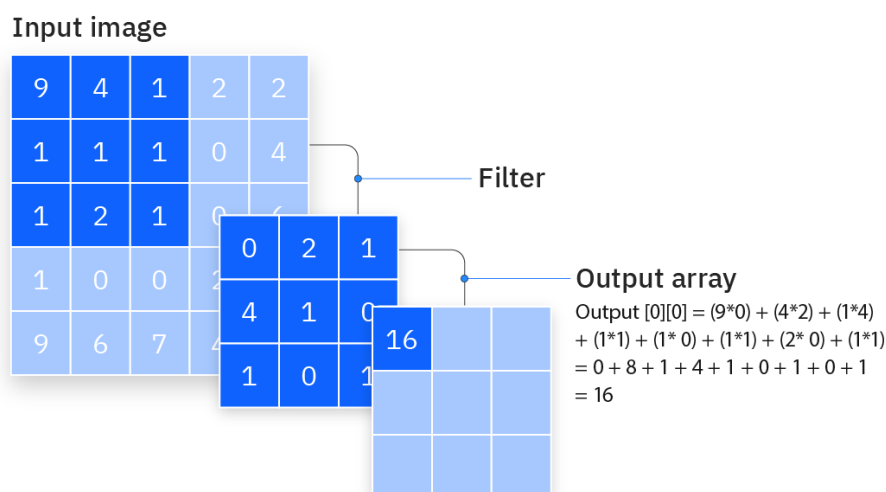


Figura 5 – Camada convolucional (IBM Inc., 2024)

Redes neurais convolucionais também costumam utilizar camadas de *pooling* após cada camada convolucional. Essa camada utiliza um método de subamostragem para reduzir a dimensão de seu *input*. Suas duas formas mais comuns são o *max pooling* e o *average pooling*.

A ideia do *pooling* é dividir o *input* em janelas não sobrepostas e transformar cada janela em um único valor escalar, aplicando sobre a janela o cálculo da média ou extraindo seu valor máximo, a depender da técnica utilizada.

As camadas convolucionais possuem a tarefa de extrair atributos da imagem, e são ótimos em identificar atributos específicos independente da região da imagem em que estejam localizados. Após a extração dos atributos, porém, ainda é necessário utilizá-los de alguma forma. Em tarefas de classificação de imagens é muito comum alimentar os atributos extraídos para camadas *fully connected*, que aprendem padrões dos atributos e dos relacionamentos entre eles e utilizam isso para classificar a imagem.

A forma padrão de um modelo de redes neurais convolucionais para classificação de imagens, portanto, é uma sequência de camadas convolucionais e camadas de *pooling*, seguidas por uma sequência de camadas *fully connected* que culminam no *output* do modelo. A figura 6 mostra um diagrama que exemplifica a arquitetura de uma rede neural convolucional para classificação de imagens.

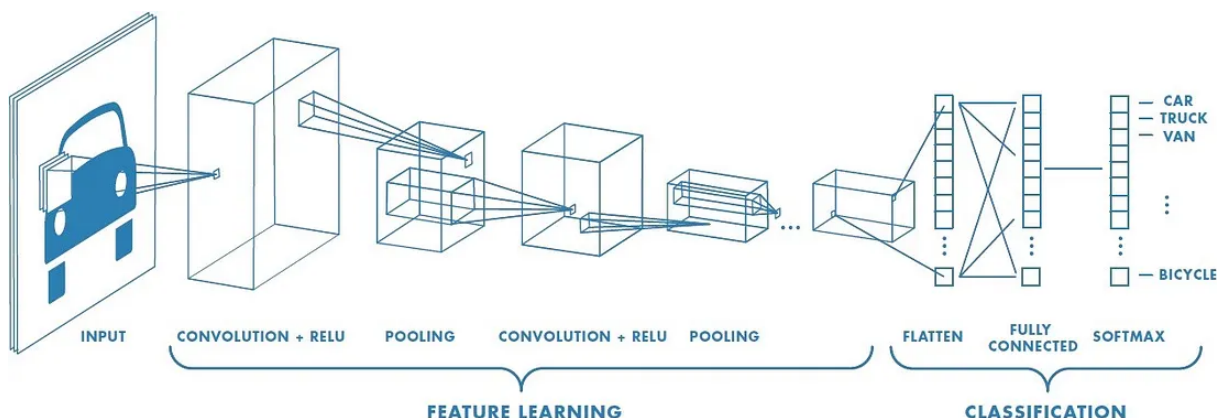


Figura 6 – Rede neural convolucional para classificação de imagens (Saha, 2018)

2.4 MODELOS DE DETECÇÃO DE OBJETOS

A tarefa de detecção de objetos é um campo desafiador dentro da área mais ampla de visão computacional. Essa tarefa consiste em identificar objetos específicos dentro de uma imagem, incluindo sua classe e localização dentro da imagem. É, portanto, uma tarefa mais complexa que a classificação de imagens, que se preocupa simplesmente em atribuir uma classe à imagem toda, enquanto a detecção de objetos deve extrair da imagem regiões retangulares chamadas *bounding boxes* e atribuir uma classe a cada uma dessas regiões.

Isso permite diversas aplicações como:

- Rastrear a movimentação de um objeto ao longo de um vídeo
- Verificar quantas instâncias de um dado objeto existem na imagem
- Extrair da imagem original uma nova imagem contendo apenas o objeto de interesse

A figura 7 exemplifica a tarefa de detecção de objetos em visão computacional.

Os modelos mais comuns de detecção de objetos costumam ser divididos em duas categorias: detectores de duas etapas e detectores de uma etapa (Zou et al., 2019). Os modelos de duas etapas primeiro dividem a imagem em regiões candidatas, onde a probabilidade de haver objetos é alta, e posteriormente gera as *bounding boxes* e classifica as regiões. Modelos de duas etapas costumam ser mais precisos, mas o tempo de inferência é mais alto.

Modelos de uma etapa, por outro lado, olham para a imagem inteira e tentam classificar as regiões e gerar as *bounding boxes* diretamente. Esses modelos possuem a vantagem de serem mais rápidos, portanto mais apropriados para aplicações de tempo real.

Como tarefas de detecção de objetos são muito utilizadas em aplicações de tempo real, a velocidade de inferência é uma métrica muito importante na avaliação

desse tipo de modelo, além da precisão do resultado. Nessa linha, modelos que colocam grande ênfase na velocidade de inferência ganharam grande tração no campo da detecção de objetos. Um desses modelos (ou família de modelos) é o YOLO, do qual falaremos mais para a frente.

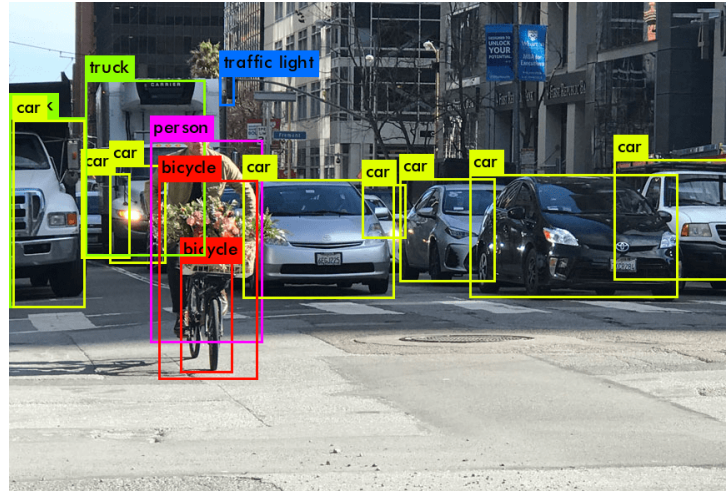


Figura 7 – Exemplo de detecção de objetos

2.4.1 Modelos YOLO

Os modelos da família YOLO (*You Only Look Once*) são modelos muito rápidos de detecção em uma etapa, portanto apropriados para aplicações de tempo real. Quando os modelos YOLO surgiram, apenas modelos de duas etapas eram utilizados para detecção de objetos com redes neurais, mas a inovação proposta com os modelos de uma etapa se mostrou extremamente eficiente e se tornou amplamente utilizada (Redmon et al., 2016).

Modelos YOLO tratam a tarefa de detecção de objetos como uma única tarefa de regressão, diferente dos modelos de duas etapas que se baseiam em uma *pipeline* mais complexa. Ele funciona dividindo a imagem em um grid $S \times S$, e cada célula do grid é responsável por detectar os objetos cujo centro está dentro da célula. Cada célula então prevê B *bounding boxes*, sendo que cada *bounding box* é representada por 5 valores: As coordenadas (x, y) do centro relativo à célula, a altura, a largura e a confiança. A confiança representa a *IoU* (*Intersection over Union*) da *bounding box* prevista com a verdadeira caso haja um objeto na célula. Se não houver objeto a confiança deve ser igual a zero (Redmon et al., 2016).

$$IoU = \frac{\text{Area de Interceccao}}{\text{Area de Uniao}} \quad (9)$$

Cada célula também produz C probabilidades de classe, cada uma representando a probabilidade de aquela célula conter um objeto de uma dada classe.

A figura 8 mostra uma representação do funcionamento da rede YOLO.

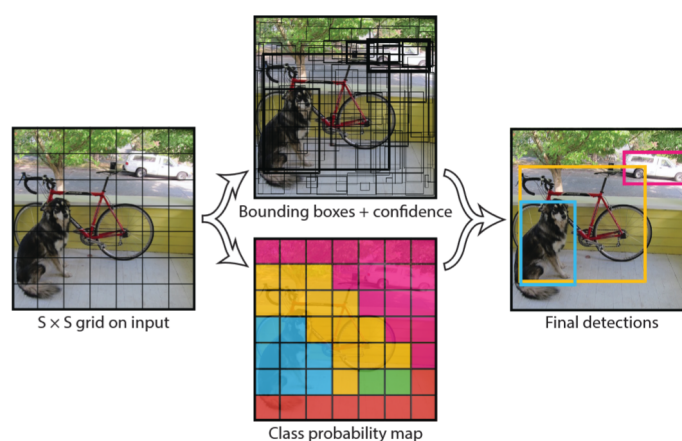


Figura 8 – Diagrama de funcionamento do modelo YOLO (Redmon et al., 2016)

2.4.2 Detecção de pontos chave

Dentro da área da detecção de objetos à também uma subdivisão chamada detecção de pontos chave. Essa tarefa consiste em, além de detectar um objeto, detectar também a posição de uma série de pontos importantes do objeto. Esses pontos podem ser importantes em vários contextos pois oferecem informação adicional a respeito da posição ou formato do objeto.

Uma aplicação muito comum da detecção de pontos chave é estimar a posição de seres humanos. Ao identificar a posição de diversas partes do corpo como ombros, cotovelos, mãos, etc. relativas umas às outras, é possível inferir a posição da pessoa e se ela está tomando certas ações, como andar ou sentar.

A figura 9 mostra um exemplo de detecção de uma série de pontos chave no corpo humano.

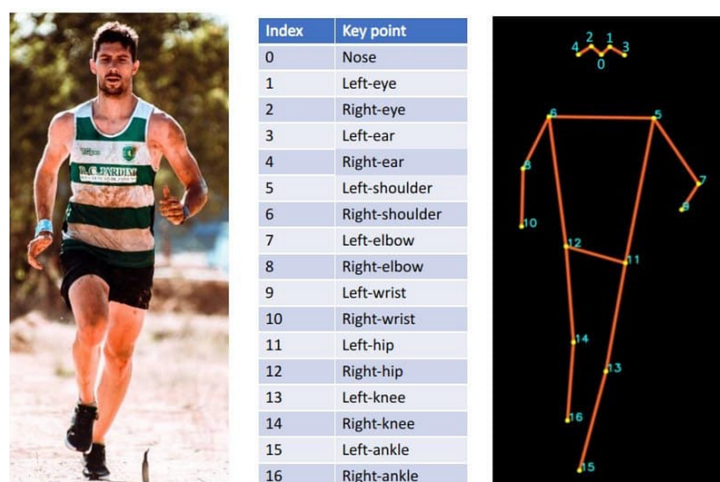


Figura 9 – Exemplo de detecção de pontos chave em um corpo humano

2.5 TÉCNICAS DE TRANSFORMAÇÃO DE IMAGEM PARA VISÃO COMPUTACIONAL

Inúmeros algoritmos de manipulação de imagens são utilizados em visão computacional desde muito antes do surgimento das redes neurais. Alguns desses algoritmos ainda são extremamente relevantes, especialmente no tratamento de imagens que possuem padrões conhecidos.

Essas técnicas são úteis em especial para extrair características das imagens e prepará-las para serem alimentadas por modelos de inteligência artificial, removendo ruídos e destacando as características mais relevantes da imagem.

As principais técnicas utilizadas nesse projeto foram:

- Transformação de perspectiva
- Limiarização adaptativa
- Segmentação de caracteres
- Filtragem de componentes conexos

2.5.1 Transformação de perspectiva

A transformação de perspectiva é uma operação utilizada para corrigir a perspectiva de uma figura 2D dentro de uma imagem. Ela permite que objetos inclinados ou rotacionados sejam alinhados com base nas coordenadas dos quatro cantos do objeto. Esse tipo de transformação não preserva paralelismo, comprimento e ângulo, mas preserva colinearidade e incidência (TheAILearner, 2023).

A transformação de perspectiva pode ser expressada como:

$$\begin{bmatrix} t_j x' \\ t_j y' \\ t_j \end{bmatrix} = \begin{bmatrix} a_1 & a_2 & b_1 \\ a_3 & a_4 & b_2 \\ c_1 & c_2 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \quad (10)$$

Sendo que x' e y' são os pontos transformados, ou seja, as coordenadas (x, y) dos pontos na imagem de saída, enquanto x e y são os pontos originais. O termo h_i é um fator de escala.

A matriz de transformação

$$M = \begin{bmatrix} a_1 & a_2 & b_1 \\ a_3 & a_4 & b_2 \\ c_1 & c_2 & 1 \end{bmatrix} \quad (11)$$

é formada por três componentes: A matriz $\begin{bmatrix} a_1 & a_2 \\ a_3 & a_4 \end{bmatrix}$, responsável por definir transformações como rotação e escala, o vetor $\begin{bmatrix} b_1 \\ b_2 \end{bmatrix}$, responsável por definir a translação e o vetor de projeção $\begin{bmatrix} c_1 & c_2 \end{bmatrix}$.

Como a matriz de transformação possui 8 incógnitas, são necessárias 8 equações para que seja resolvida. Para isso são selecionados quatro pontos (x, y) na imagem original e quatro pontos (x', y') na imagem de saída. Dessa forma é possível obter a matriz de transformação capaz de mapear todos os pontos da imagem original para a imagem final.

A figura 10 exemplifica o que ocorre com uma imagem na transformação de perspectiva.

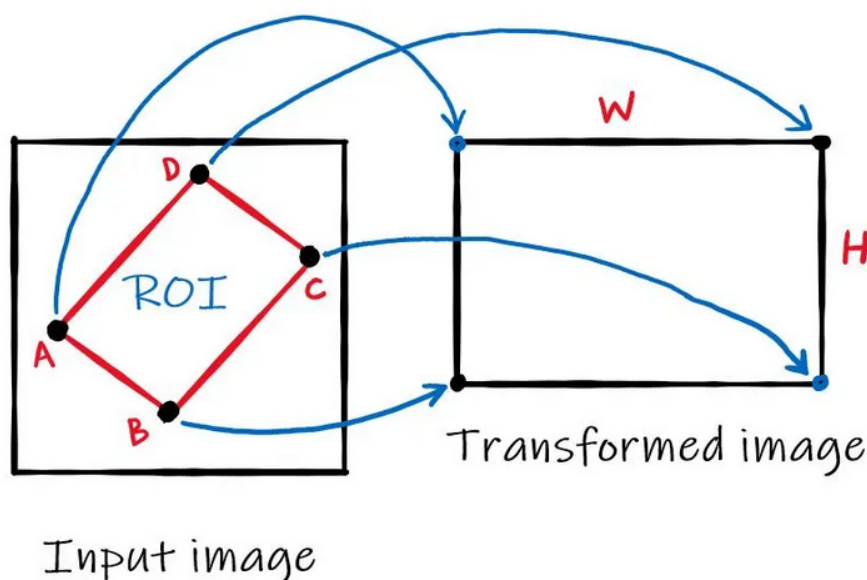


Figura 10 – Transformação de perspectiva (TheAILearner, 2023)

2.5.2 Limiarização adaptativa

A limiarização é uma técnica muito utilizada em visão computacional para criar máscaras a partir de imagens. Ela consiste em definir um limiar e classificar todos os pixels da imagem em 0 (preto) ou um (branco), dependendo se o valor original do pixel ficava acima ou abaixo do limiar definido. Uma aplicação muito comum desse conceito em visão computacional é a separação de um objeto do fundo da imagem.

Independente da aplicação, a escolha do valor do limiar impacta profundamente a imagem binarizada resultante. Uma escolha mal feita pode fazer com que partes significativas do objeto sejam apagadas ou com que ruídos do fundo se misturem ao objeto. Portanto, a aplicação de uma boa técnica para selecionar o limiar se faz necessária.

Um outro fator relevante a se considerar é que em muitos casos a iluminação faz com que certas partes da imagem sejam mais escuras que outras. Isso faz com que seja interessante utilizar um limiar diferente em diferentes partes da imagem, de forma a evitar distorções por diferença na iluminação.

Um método para determinação desse limiar variável é o método de Sauvola (Sauvola; Pietikäinen, 2000), que determina um limiar ideal para cada pixel da imagem usando os pixels vizinhos como base.

O método funciona calculando a média e desvio padrão local para cada pixel com base em uma janela. A partir disso é então calculado o limiar ideal para aquele pixel.

A média local para um dado pixel de coordenadas (x, y) na imagem I é definida como:

$$\mu(x, y) = \frac{1}{W^2} \sum_{i=-\frac{W}{2}}^{\frac{W}{2}} \sum_{j=-\frac{W}{2}}^{\frac{W}{2}} I(x + i, y + j) \quad (12)$$

Sendo que W é o tamanho da janela escolhido.

Já o desvio padrão local desse mesmo pixel é definido como:

$$\sigma(x, y) = \sqrt{E[I^2](x, y) - \mu(x, y)^2} \quad (13)$$

Sendo que $E[I^2](x, y)$ é a média local do pixel de coordenadas (x, y) na imagem I^2 .

Por fim, o Limiar de Sauvola do pixel de coordenadas (x, y) na imagem I é:

$$T(x, y) = \mu(x, y) * [1 + k(\frac{\sigma(x, y)}{R} - 1)] \quad (14)$$

Sendo que R é o valor máximo teórico de σ e k é um parâmetro de sensibilidade, tipicamente 0.3.

2.5.3 Segmentação de caracteres

A segmentação de caracteres serve para dividir uma imagem contendo múltiplos caracteres em um grupo de imagens que contém apenas um caractere em cada. Partindo do pressuposto que a imagem original é binarizada, contendo portanto apenas pixels com valores de zero ou um, a segmentação de caracteres pode ser feita a partir da detecção de espaços brancos entre os caracteres (Casey; Lecolinet, 1996).

2.5.4 Filtragem de componentes conexos

A filtragem de componentes conexos é um processo que pode ser usado para isolar o objeto de interesse em uma imagem binarizada e remover ruído.

Ao binarizar uma imagem é comum que pixels que não pertencem ao objeto analisado acabem sendo classificados como parte do objeto, o que pode atrapalhar a análise da imagem. As vezes esses pixels estão conectados ao objeto, o que torna mais difícil identificá-los, mas as vezes estão desconectados, o que facilita sua identificação. Quando os pixels de ruído estão desconectados do objeto é possível identificá-los ao separar a imagem em *blobs*, regiões isoladas umas das outras.

Cada *blob* representa uma região da imagem na qual todos os pixels possuem valor 1 e possuem conectividade com pelo menos um outro pixel que também pertence àquele *blob*. A figura 11 exemplifica a separação de regiões da imagem em *blobs* conexos. Cada cor representa um *blob* diferente.



Figura 11 – Exemplo de separação de regiões por componentes conexos

A imagem separada em *blobs* pode ser representada como uma matriz na qual a cada posição é atribuído um número a depender do *blob* ao qual àquela posição pertence. Esses números são os rótulos das regiões.

Para realizar a filtragem, então basta determinar qual é o *blob* com maior área e eliminar os demais. A área de um *blob* nesse caso nada mais é do que a quantidade de pixels pertencentes à ele.

2.6 NEURAL PROCESSING UNIT

A unidade de processamento NPU (*Neural Processing Unit*) é um processador especializado em redes neurais. Ele é otimizado para operações matriciais e convolucionais, que são algumas das operações mais comuns nas redes neurais.

O intuito da NPU é permitir a execução de modelos de redes neurais de forma eficiente em aplicações embarcadas, que não costumam ser munidas de CPUs ou

GPUs poderosas. Para isso a NPU sacrifica flexibilidade em prol da performance em uma tarefa específica.

A arquitetura de uma NPU consiste basicamente em um grid de *processing engines* (PE), sendo que cada uma performa a computação de um neurônio (multiplicação, acumulação e sigmoide). A figura 12 mostra a arquitetura da NPU (Chen et al., 2020).

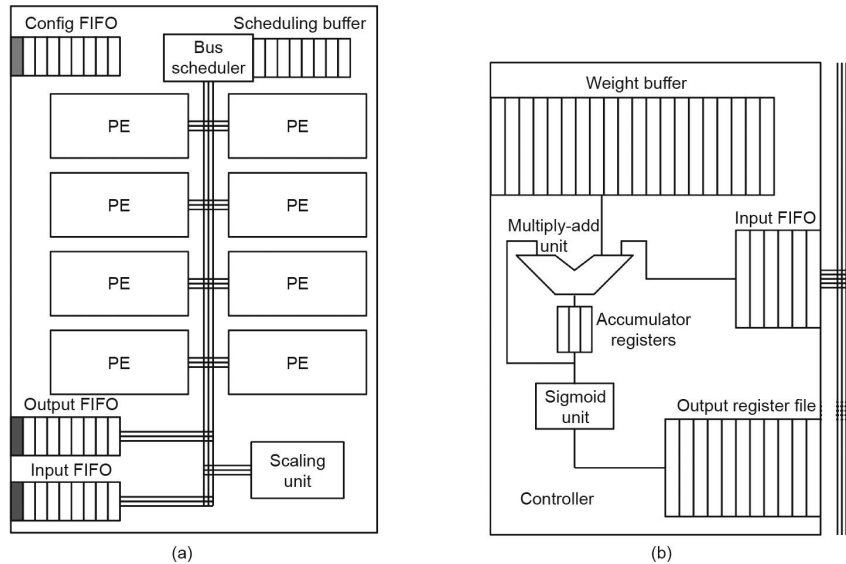


Figura 12 – A NPU. (a) NPU de 8 *processing engines*; (b) Fila de uma *processing engine* (Chen et al., 2020)

2.7 QUANTIZAÇÃO DE REDES NEURAIS

No contexto de redes neurais artificiais a quantização é o processo de converter os pesos e ativações da rede, que originalmente são valores reais (normalmente de 32 *bits*), para valores inteiros, normalmente de 8 *bits*. Esse processo reduz a precisão numérica, mas pode gerar ganhos significativos de performance, além de reduzir o espaço em memória ocupado pelo modelo (Jacob et al., 2018).

Em um possível esquema de quantização proposto por (Jacob et al., 2018) a relação entre um valor quantizado q e um valor real r é definida como:

$$r = S(q - Z) \quad (15)$$

Onde S e Z são os parâmetros de quantização, que são diferentes para cada vetor de pesos e ativações da rede. O parâmetro Z (*zero-point*) representa o valor de q quando $r = 0$, permitindo que o erro da quantização seja nulo para $r = 0$. Já a constante S (*scale*) é um valor real positivo.

É possível realizar a quantização após o treinamento do modelo, mas em muitos casos é recomendado que o modelo seja treinado já aplicando uma forma de quantização simulada, para que aprenda a compensar os erros de quantização.

Para se obter o valor quantizado q dado um valor real r é aplicado o seguinte algoritmo:

$$\text{clamp}(r; a, b) := \min(\max(r, a), b) \quad (16)$$

$$s(a, b, n) := \frac{b - a}{n - 1} \quad (17)$$

$$q(r; a, b, n) := \left\lfloor \frac{\text{clamp}(r; a, b) - a}{s(a, b, n)} \right\rfloor s(a, b, n) + a \quad (18)$$

Onde:

- r é o valor real sendo quantizado
- $[a; b]$ é o *range* de quantização, os valores mínimo e máximo esperados para r
- n é o número de níveis de quantização (256 para 8 *bits*)
- $\lfloor . \rfloor$ representa arredondamento para o inteiro mais próximo

Quantizando desta forma é possível obter um bom balanço entre performance e acurácia do modelo (Jacob et al., 2018).

3 DESCRIÇÃO DO PROBLEMA E REQUISITOS TÉCNICOS

Neste capítulo será discutido de forma mais aprofundada o problema que esse projeto se propõe a resolver. Isso envolve, além de uma observação detalhada do problema e seus requisitos, o contexto no qual o problema está inserido dentro da empresa e as motivações para resolvê-lo.

3.1 DESCRIÇÃO DA EMPRESA

A Palmsoft Tecnologia é uma empresa de desenvolvimento de software sob demanda. Produz sites, aplicativos, jogos, soluções de inteligência artificial e outros produtos de software.

Um dos produtos desenvolvidos pela Palmsoft é o *Aegis Access Control*. Uma plataforma que gerencia controle de acesso em condomínios equipados com fechaduras eletrônicas de desbloqueio por imagem facial.

O objetivo do *Aegis Access Control* é atender proprietários e gestores de apartamentos que fazem aluguel de curto prazo, permitindo que eles automatizem o processo de concessão de permissão de acesso aos imóveis. Quando o hóspede faz uma reserva, o gestor do apartamento acessa uma interface *web* onde ele cadastra o hóspede e as informações da reserva, como data de início e de fim. O hóspede então recebe um *email* com um *link* que o leva à uma página onde ele deve cadastrar uma foto do seu rosto, além de outras informações complementares, como placa do carro. Com base nas informações da reserva, o sistema então libera o acesso do hóspede ao condomínio e ao apartamento específico que foi alugado, apenas durante o período da estadia. Em cada porta do condomínio há um controlador de acesso facial, que destrava a fechadura da porta com base no rosto do usuário. O cadastro do rosto nos controladores de acesso é feito utilizando a foto enviada pelo usuário.

Para completar a experiência, o estacionamento é equipado com uma câmera com leitura automática de placas integrada, conectada ao portão. Da mesma forma que as imagens faciais são enviadas pelo sistema aos controladores de acesso das portas, as placas dos carros dos hóspedes também são enviadas às câmeras. Quando o carro do hóspede se aproxima, então, a câmera lê a placa do veículo, identifica que aquela placa possui acesso ao condomínio e envia um sinal ao portão para que ele abra.

Dessa forma, o hóspede recebe uma experiência completamente automatizada. Chega no condomínio e seu acesso já está liberado sem que precise ter contato com nenhum humano.

Além da experiência do hóspede, essa tecnologia é muito benéfica para os gestores dos apartamentos, que conseguem administrá-los de forma remota. Quando precisam conceder acesso a um prestador de serviço que vai limpar o apartamento,

por exemplo, isso também é feito com o mesmo processo.

3.2 MOTIVAÇÃO

Dentro do contexto do condomínio autônomo, as câmeras instaladas na garagem são um fator importante, que adiciona conforto à experiência do cliente. Essas câmeras são equipadas com tecnologia de visão computacional capaz de ler a placa de um veículo em questão de milissegundos, com ótima precisão.

Câmeras como essa já são parte do dia a dia de muitas pessoas, em diversas aplicações. Uma delas, muito comum, é a de controle de acesso em estacionamentos pagos, como em shoppings ou supermercados. Elas permitem que o sistema registre a placa do carro de cada cliente e a associe ao seu *ticket* de estacionamento. Uma das vantagens disso é que caso o cliente perca o *ticket* impresso o sistema poderá gerar um novo sem problemas, mantendo o mesmo horário de entrada. Outra vantagem é que quando o cliente se aproxima da cancela de saída a placa do seu carro é rapidamente detectada e o sistema pode conferir se aquele cliente pagou o estacionamento sem que ele precise utilizar o *ticket* impresso na saída. Isso evita congestionamentos na saída e melhora o fluxo de veículos ao poupar tempo. Esse sistema também pode ser utilizado para conferir acesso gratuito à funcionários, permitindo a entrada deles sem retirada de *ticket* com base na placa do carro.

Outra aplicação para esse tipo de câmera é em sistemas de segurança em estacionamentos. Ao detectar placas de carros o sistema pode gerar *logs* de entrada e saída de veículos. Isso ajuda muito na auditoria dos vídeos das câmeras de segurança do estacionamento, pois os momentos mais relevantes ficam registrados. Dessa maneira fica fácil responder perguntas como: "Em qual data e hora o carro com a placa X acessou o estacionamento?". Além disso é possível identificar facilmente acesso de veículos não autorizados à um estacionamento privado.

No condomínio que utiliza a solução do *Aegis Access Control* a câmera utilizada é um dispositivo da Intelbras, modelo VIP 5460 LPR IA, cujo valor de mercado varia entre R\$ 5.700 e R\$ 7.500. O alto valor desse dispositivo motivou a exploração dessa tecnologia para desenvolvimento de alternativas mais acessíveis, que possam ser utilizadas em aplicações onde o custo é um fator determinante.

Um outro fator motivador é que a Palmsoft Tecnologia demonstrou interesse em testar a performance de *hardware* de baixo custo para aplicações de inteligência artificial, com o objetivo de validar o equipamento para desenvolvimento de produtos nessa área. Para isso, foi adquirida uma placa Rock 3A, da radxa, equipada com uma NPU de 0.8 TOPs.

Portanto, juntando a motivação do desenvolvimento de uma alternativa barata para câmera com leitura automática de placas de carro com a de validar uma placa de baixo custo para aplicações de inteligência artificial, foi pensado um projeto que

desenvolvesse um modelo de leitura de placas de carro e validasse sua execução na placa Rock 3A.

O objetivo desse projeto, portanto, não é criar um protótipo funcional, mas criar e validar um algoritmo de leitura de placas de carro, que recebe uma imagem e retorna os caracteres da placa do veículo contido na imagem (se houver), e analisar a performance desse algoritmo ao ser executado na placa Rock 3A, validando assim a possibilidade da criação de um produto que utilize o algoritmo criado nesse *hardware*.

3.3 ANÁLISE DO PROBLEMA

O objetivo geral deste projeto pode ser definido como:

"Desenvolver um software de leitura automática de placas de veículos e validar a viabilidade funcional de uma solução de baixo custo que execute esse software em uma placa Rock 3A."

O problema, portanto pode ser dividido em duas partes:

- Desenvolver o *software* de leitura automática de placas de veículos
- Validar a placa Rock 3A como um *hardware* viável para execução desse *software*

A primeira parte envolve o desenvolvimento de um algoritmo complexo de visão computacional capaz de ler os caracteres da placa de um carro ao longo de múltiplas etapas, exigindo conhecimento técnico acerca dos diferentes modelos de inteligência artificial para processamento de imagens e técnicas de visão computacional.

Já a segunda parte envolve executar e avaliar a performance do algoritmo no *hardware*. Para isso é necessária compreensão do dispositivo a ser utilizado, os requisitos para executar modelos de redes neurais na NPU e as configurações necessárias.

3.3.1 Criação do algoritmo de leitura automática de placas de carros

O *software* de leitura automática de placas de veículos a ser desenvolvido nada mais é do que um algoritmo que recebe uma imagem como entrada e devolve uma sequência de caracteres correspondente à placa do carro contido na imagem (se houver). Esse algoritmo deve ser possível de ser executado na placa Rock 3A, portanto deve se observar as limitações desse *hardware* que serão discutidas posteriormente.

As características desejáveis do algoritmo são principalmente:

- **Acurácia:** O algoritmo deve inferir corretamente a maioria das placas dos carros contidos nas imagens entregues a ele. Um leitor automático de placas que comete muitos erros não tem utilidade, portanto a proporção de placas inferidas

corretamente em relação ao total de imagens analisadas deve ser o mais alta possível.

- **Velocidade:** Para muitas aplicações a velocidade é crítica, especialmente quando se deseja inferir a placa de carros passantes. Portanto, o tempo necessário para que o algoritmo processe uma imagem e retorne o resultado deve ser mínimo.

3.3.2 Teste do algoritmo desenvolvido na placa Rock 3A

Para realizar a etapa de testes do algoritmo na placa Rock 3A é necessário entender as características e limitações desse dispositivo. Além disso, escolhas devem ser feitas na etapa de desenvolvimento do algoritmo considerando as características dessa placa, para que não haja problemas de incompatibilidade.

A placa Rock 3A é um computador de placa única (SBC - Single Board Computer) desenvolvida pela Radxa, projetada para aplicações de inteligência artificial e processamento de imagens. Esta placa foi escolhida para validação devido ao seu custo acessível e às suas capacidades de processamento de IA.

As principais especificações técnicas da Rock 3A incluem:

- **Processador:** Rockchip RK3568 quad-core ARM Cortex-A55 de até 2.0GHz
- **NPU:** Unidade de Processamento Neural de 0.8 TOPs (Tera Operations Per Second)
- **Memória RAM:** Disponível em versões de 2GB, 4GB e 8GB LPDDR4
- **Armazenamento:** Slot para cartão microSD e conector eMMC
- **Conectividade:** Wi-Fi 802.11 b/g/n/ac, Bluetooth 5.0, Ethernet Gigabit
- **Interfaces:** Múltiplas portas USB, HDMI, GPIO de 40 pinos
- **Suporte a câmeras:** Interface MIPI CSI para conexão de câmeras

A presença da NPU (*Neural Processing Unit*) de 0.8 TOPs é particularmente relevante para este projeto, pois permite a aceleração de operações de redes neurais, potencialmente melhorando significativamente a performance do algoritmo de leitura de placas em comparação com processamento puramente baseado em CPU.

As limitações da placa que devem ser consideradas no desenvolvimento incluem:

- **Capacidade de processamento limitada:** Comparada a soluções mais caras, a Rock 3A possui recursos computacionais limitados

- **Memória restrita:** Dependendo da versão, a quantidade de RAM pode ser um fator limitante para modelos de IA mais complexos
- **Compatibilidade de software:** Nem todos os frameworks de IA possuem suporte otimizado para a arquitetura ARM e NPU específica

A escolha da Rock 3A para este projeto se justifica pela necessidade de validar uma solução de baixo custo que possa ser viável comercialmente, mantendo um equilíbrio entre performance e custo-benefício para aplicações de leitura automática de placas veiculares.

3.3.2.1 *Neural Processing Unit* da placa Rock 3A

A NPU da Rock 3A é otimizada para operações de inferência, não sendo adequada para treinamento de modelos. Ela oferece melhor performance quando utilizada com modelos quantizados para *uint8*, formato que reduz significativamente o uso de memória e acelera as operações matemáticas, mantendo uma precisão aceitável para a maioria das aplicações.

Para utilizar a NPU da Rock 3A, é necessário trabalhar com o *framework* RKNN (*Rockchip Neural Network*), que é o SDK (*Software Development Kit*) oficial fornecido pela Rockchip para desenvolvimento de aplicações de inteligência artificial em seus processadores. O RKNN oferece ferramentas para conversão de modelos treinados em *frameworks* populares como TensorFlow, PyTorch e ONNX para o formato nativo da NPU.

Os principais requisitos para executar modelos de redes neurais na NPU da Rock 3A incluem:

- **Conversão de formato:** Os modelos devem ser convertidos para o formato RKNN utilizando o *toolkit* oficial
- **Quantização:** Os modelos devem ser quantizados para *uint8* para otimizar a performance na NPU
- **Compatibilidade de operações:** Nem todas as operações de redes neurais são suportadas pela NPU, sendo necessário verificar a compatibilidade das camadas utilizadas
- **Limitações de memória:** O modelo convertido deve respeitar as limitações de memória da NPU
- **Formato de entrada:** As imagens de entrada devem estar no formato e resolução adequados para o modelo convertido

A utilização da NPU oferece vantagens significativas em termos de eficiência energética e velocidade de inferência quando comparada ao processamento puramente baseado em CPU, tornando-a adequada para aplicações que requerem processamento de imagens em tempo real com restrições de energia e custo.

3.4 REQUISITOS DA SOLUÇÃO

Considerando as características e especificações do projeto discutidas até então, os requisitos funcionais e não funcionais do projeto podem ser definidos como:

3.4.1 RF1: *Software* de Detecção Automática de Placas

O *software* deve ter a capacidade de receber imagens de automóveis como carros, caminhões e ônibus, analisá-las e retornar os caracteres da placa do veículo contido na imagem.

3.4.1.1 Requisitos não funcionais:

Código	Nome	Descrição
NF1.1	Acurácia	O percentual de placas inferidas corretamente deve ser suficiente para que a ferramenta seja útil. Preferencialmente acima de 80%.
NF1.2	Velocidade	O tempo necessário para realizar a inferência deve ser apropriado para uma aplicação de tempo real, não maior que 1 segundo por imagem.
NF1.3	Flexibilidade	O sistema deve funcionar tanto para placas do modelo antigo, quanto para placas do modelo novo (Mercosul), abrangendo todas as variações possíveis dentro destes modelos.
NF1.4	Especificidade	O algoritmo deve ser capaz de identificar qual é o modelo da placa, entre o modelo antigo e o novo (Mercosul).
NF1.3	Compatibilidade	O <i>software</i> deve ser compatível com a placa Rock 3A.

Tabela 1 – Requisitos não funcionais de RF1

3.4.2 RF2: Execução na Placa Rock 3A

O algoritmo de leitura automática de placas de veículos deve ser executado na placa Rock 3A da Radxa.

3.4.2.1 Requisitos não funcionais:

Código	Nome	Descrição
NF2.1	<i>Neural Processing Unit</i>	A execução dos modelos de redes neurais na placa Rock 3A deve ser realizada na NPU.
NF2.2	Quantização	Os modelos executados na NPU devem ser quantizados para <i>uint8</i> para otimizar o tempo de inferência.

Tabela 2 – Requisitos não funcionais de RF2

4 SOLUÇÃO PROPOSTA E METODOLOGIA UTILIZADA

Neste capítulo será discutida a solução que foi avaliada como a mais apropriada para resolução do problema do projeto.

O primeiro ponto a ser considerado é a metodologia de desenvolvimento da solução como um todo, vista como um problema de mineração de dados. Além disso, é feita desde uma análise detalhada dos dados utilizados e do objeto de interesse (placas de veículos), ao detalhamento do algoritmo de leitura de placas de carro que foi desenvolvido.

Também são discutidas as soluções utilizadas para adaptação do algoritmo visando a sua execução na placa Rock 3A, incluindo as tecnologias e processos necessários.

Por fim, é considerada a metodologia de teste da solução, os passos necessários para evitar viés nos resultados e garantir relevância estatística para os números obtidos.

4.1 METODOLOGIA DE DESENVOLVIMENTO

A metodologia de desenvolvimento da solução como um todo segue o padrão CRISP-DM, discutido no capítulo 2, que trata o projeto como um problema de mineração de dados. Nesse padrão, o primeiro passo para o desenvolvimento do projeto é o entendimento do problema de negócio, que já foi discutido e bem definido no capítulo 3.

As etapas do projeto definidas a permitir do padrão CRISP-DM são portanto:

1. Entendimento do problema

O problema de negócio foi bem definido e explorado no capítulo 3, tendo como objetivo geral *"Desenvolver um software de leitura automática de placas de veículos e validar a viabilidade funcional de uma solução de baixo custo que execute esse software em uma placa Rock 3A."*

2. Entendimento dos dados

Para obter um bom entendimento dos dados é necessário explorar as características tanto do *dataset* utilizado quanto do objeto de interesse a ser analisado, as placas de veículos. Esse estudo será feito na próxima seção deste capítulo.

3. Preparação dos dados

A preparação dos dados envolve duas etapas distintas, pois esse projeto lida com dois conjuntos de dados, sendo o segundo derivado do primeiro. O primeiro conjunto são as imagens dos carros e anotações das placas, e a preparação desses dados envolve separar o *dataset* em conjuntos de treino, validação e

teste, além de organizá-los e transformá-los para que fiquem no padrão COCO (Lin et al., 2014). Essa organização e transformação é necessária para que o modelo de detecção de placas seja treinado.

A segunda etapa da preparação dos dados ocorre posteriormente e envolve a criação de um segundo conjunto de dados gerado a partir do primeiro. Esse segundo conjunto de dados deve ser um *dataset* de imagens de caracteres de placas de carro e também um *dataset* de imagens de placas de carro inteiras, voltados para o treinamento de modelos de classificação de caracteres. Para realização dessa tarefa, porém, é necessário que já tenha havido um avanço na modelagem, de forma que a etapa de preparação dos dados e de modelagem possuem alternância.

4. Modelagem

A modelagem, por sua vez, pode ser dividida em três etapas principais. A primeira é a seleção e treinamento de um modelo de detecção de objetos para a detecção das placas, etapa essa que depende da preparação do primeiro conjunto de dados, contendo as imagens dos carros e anotações das placas.

A segunda etapa, que depende da primeira, é o desenvolvimento de um algoritmo para extração dos caracteres das placas identificadas pelo modelo de detecção de objetos, utilizando técnicas clássicas de visão computacional. Após essa etapa é possível (e necessária) a realização da segunda parte da preparação dos dados, que gera o *dataset* de caracteres extraídos.

Por fim, a última etapa de modelagem é a criação e treinamentos de modelos de redes neurais de classificação de caracteres, que devem ser capazes de identificar caracteres a partir das imagens extraídas pelo algoritmo de extração de caracteres desenvolvido.

5. Avaliação

A avaliação envolve realizar testes com o algoritmo completo e extrair métricas de desempenho. Esses testes devem ser realizados tanto na CPU e GPU do PC quanto na CPU e NPU da placa Rock 3A, e avaliar tanto a confiabilidade do modelo em inferir corretamente as placas dos veículos quanto a velocidade de inferência.

6. Deployment

No caso deste projeto o *deployment* nada mais é do que realizar um relatório final dos resultados para validar o modelo e o *hardware* como viáveis ou não para o desenvolvimento de um produto de leitura automática de placas veiculares. Esse relatório estará contido no capítulo 5.

4.2 ANÁLISE DOS DADOS

Antes que seja possível desenvolver qualquer modelo de leitura de placas veiculares é necessário entender as características das placas a serem analisadas, além das características do *dataset* específico a ser utilizado no desenvolvimento.

Nesta seção serão exploradas as diferentes variações de placas de carro existentes no Brasil que serão encontradas na nossa análise, e também os atributos dos dados a serem utilizados, como o formato das anotações e descrição da metodologia de coleta.

4.2.1 Características do objeto

O sistema de placas de carros do Brasil já passou por diversas alterações ao longo dos anos, sendo que a última mais significativa foi a adoção do "padrão Mercosul", em 2018. Desde então, dois padrões de placas veiculares seguem coexistindo, o padrão Mercosul e o padrão anterior, de 1990, que segue em circulação embora não sejam mais emitidas novas placas nesse formato.

O padrão antigo, iniciado em 1990, é um padrão alfanumérico de 7 caracteres, que seguem o formato ABC·1234. Três letras e um número de 4 dígitos, com um ponto ou hífen separando os dois blocos.

As placas dessa modalidade costumam ser da cor cinza metálico com caracteres pretos (figura 13), mas existem exceções importantes. Nesse padrão, algumas categorias de veículos utilizam placas de fundo colorido e caracteres brancos. A mais comum dessas categorias é a placa utilizada em veículos que realizam transporte pago, cujo fundo é vermelho (figura 14).



Figura 13 – Modelo de placa veicular do Brasil de 1990



Figura 14 – Modelo de placa veicular do Brasil de 1990 para veículos que realizam transporte pago

As placas novas, do padrão Mercosul, utilizam uma sequência alfanumérica diferente. O formato é ABC1D23, que contém 4 letras, um número de 1 dígito e outro número de 2 dígitos, sendo que a quarta letra fica entre os dois números. Esse formato permite mais combinações diferentes que o anterior.

As placas do padrão Mercosul possuem sempre fundo branco, enquanto as letras podem variar em cor a depender da modalidade de veículo, sendo preto a mais comum (figura 15).



Figura 15 – Modelo de placa veicular do padrão Mercosul

Ambos modelos de placas possuem informações adicionais na parte de cima, notadamente o padrão de 1990 inclui a cidade e UF do veículo e o padrão Mercosul inclui o país. Além disso, as placas do padrão Mercosul incluem símbolos e selos adicionais que ajudam na verificação de autenticidade. Os dois modelos possuem margens ao redor do texto principal.

4.2.2 Características do *dataset* utilizado

O conjunto de dados utilizado foi RodoSol-ALPR, criado no trabalho (Laroca et al., 2022). Esse *dataset* é composto por 10.000 imagens de carros e 10.000 imagens de motocicletas com dimensão de 1.280 x 720 px. As imagens, além de serem classificadas como "carro" ou "moto", também são classificadas entre veículos com placa no padrão antigo (BR) e com placa no padrão Mercosul (ME), sendo que metade das imagens tanto de carros quanto de motos pertencem a cada padrão de placa.

As imagens desse *dataset* foram capturadas por câmeras estáticas em um pedágio na Rodovia do Sol, no Espírito Santo. Inclui imagens com iluminação boa e ruim, com chuva e sem, além de carros em diferentes distancias da câmera.

As anotações das imagens são no seguinte formato:

```
type: car  
plate: ODE2510  
layout: Brazilian  
corners: 558,438 687,439 687,482 558,481
```

A primeira linha especifica o tipo de veículo (*car/motorcycle*), a segunda, os caracteres da placa, a terceira o layout da placa (*Brazilian/Mercosur*) e a quarta as coordenadas dos 4 cantos da placa em pixels.

A figura 16 é um exemplo de imagem do *dataset* com as marcações dos 4 cantos da placa obtidas do arquivo de anotação.



Figura 16 – Exemplo de imagem do *dataset* com as marcações dos 4 cantos da placa

4.3 PIPELINE DE LEITURA DE PLACAS

O desenvolvimento do *pipeline* de leitura de placas já faz parte da etapa de modelagem do método CRISP-DM. O objetivo do *pipeline* é receber imagens de veículos e devolver uma sequência de caracteres correspondente à placa de identificação do veículo contido na imagem.

Essa tarefa não é um problema trivial de visão computacional, e dificilmente pode ser resolvida simplesmente alimentando as imagens a um modelo de redes neurais artificiais. A complexidade se dá principalmente pelos seguintes fatos:

- A placa pode estar localizada em locais variados da imagem e ter diferentes tamanhos
- A quantidade de placas possíveis é aproximadamente 4.6×10^8 para placas do padrão Mercosul e 1.8×10^8 para placas no padrão antigo, sendo impossível treinar um modelo no qual cada placa seja tratada como uma classe distinta
- A maior parte do conteúdo de cada imagem é apenas ruído que não pode ser aproveitado para a leitura da placa

Por conta da complexidade, o problema requer uma solução refinada. Com base nisso, o *pipeline* proposto para a leitura de placas pode ser dividido em três blocos sequenciais:

1. Detecção e classificação da placa

O bloco de detecção e classificação da placa deve extrair da imagem de entrada a localização exata da imagem, permitindo que todo o resto da imagem possa ser descartado na etapa seguinte. Além disso, esse bloco deve classificar a placa entre os modelos Brasil e Mercosul e identificar as coordenadas cartesianas dos quatro cantos da placa.

2. Processamento da placa

Este bloco deve realizar transformações na imagem utilizando as informações obtidas pelo modelo de detecção. Essas transformações envolvem cortar a imagem, para remover tudo o que não seja a placa, e também outras operações de processamento que facilitem a leitura dos caracteres no bloco seguinte.

3. Leitura da placa

Por fim, o último bloco do *pipeline* recebe a imagem processada e aplica o algoritmo de leitura para efetivamente extrair os caracteres da placa.

A seguir serão exploradas as soluções propostas para cada bloco.

4.3.1 Modelo de detecção de objetos e pontos-chave

Para a tarefa de detecção e classificação da placa, a solução proposta foi utilizar um modelo de detecção de objetos da família YOLO, mais especificamente o YOLO26-pose, desenvolvido pela Ultralytics.

Os modelos da família YOLO são notórios por serem precisos e rápidos na detecção de objetos. Nessa linha, os modelos YOLO26 foram desenvolvidos pensando especificamente em IA de borda, sendo portanto otimizados para entregar excelente performance com baixa latência e utilizando o mínimo de recursos.

Uma das variantes disponíveis do YOLO26 é o YOLO26-pose, que além de realizar detecção de objetos também realiza detecção de pontos-chave na imagem, sendo portanto ideal para o caso de uso desse projeto.

O YOLO26-pose ainda possui 5 tamanhos, apresentados na tabela 3.

Tamanho	Parâmetros (milhões)	Velocidade CPU (ms)	mAPpose 50-95(e2e)
<i>nano</i>	2.9	40.3	57.2
<i>small</i>	10.4	85.3	63.0
<i>medium</i>	21.5	218.0	68.8
<i>large</i>	25.9	275.4	70.4
<i>extra large</i>	57.6	565.4	71.6

Tabela 3 – Tamanhos do modelo de detecção de objetos YOLO26-pose

A métrica mAPpose 50-95(e2e) representa a precisão média para estimativa de pose (pontos chave). Nota-se que quanto maior o modelo mais alta é a precisão, porém mais alta também é a latência. Para este projeto, como estamos lidando com um objeto relativamente simples (placas de carro) e o *hardware* é bastante limitado, foi escolhida a versão *nano* do modelo, que prioriza velocidade de inferência e economia de recursos.

4.3.2 Algoritmo de processamento da imagem

Uma vez detectada a placa, incluindo as coordenadas dos seus 4 cantos na imagem, torna-se possível realizar diversas transformações na imagem para facilitar a tarefa de leitura dos caracteres da placa. Essas transformações devem ter como objetivo extrair o máximo de informação e remover o máximo de ruído possível, de forma a entregar para a etapa seguinte dados mais limpos, informativos e padronizados.

Com esse objetivo, a sequência proposta de transformações é a seguinte:

1. Conversão da imagem para escala de cinza
2. Extração da placa com base na *bounding box*
3. Transformação de perspectiva
4. Remoção das bordas
5. Binarização
6. Separação dos caracteres
7. Limpeza das imagens de caracteres

A seguir é discutida cada uma dessas etapas.

4.3.2.1 Conversão para escala de cinza

Enquanto as cores poderiam ser úteis para detectar a placa na imagem original, elas já não têm muito propósito a partir desse ponto, pois as placas praticamente não têm cores, e quando têm elas não são relevantes. Por conta disso, as cores são informação desnecessária que só complicaria as etapas posteriores, de forma que é melhor eliminá-las.

Ao converter a imagem para escala de cinza (figura 17) ela passa a ser representada por uma matriz $h \times w$ (sendo h a altura da imagem e w a largura), ao invés de três matrizes, como era quando tinha cores.



Figura 17 – Imagem em escala de cinza

4.3.2.2 Extração da placa com base na *bounding box*

A *bounding box* de um objeto em uma imagem nada mais é do que os limites superior, inferior, direito e esquerdo do objeto. Com essas quatro informações é possível formar um retângulo que, idealmente, contém o objeto inteiro sem ser maior do que o necessário.

Uma das saídas do modelo de detecção de objetos é a *bounding box* de cada objeto detectado na imagem. É importante notar que é possível que mais de um objeto seja detectado em uma dada imagem, caso no qual apenas a melhor detecção será utilizada, ou seja, a que tiver a confiança mais alta.

Com base na *bounding box* obtida e na imagem em escala de cinza é gerada uma nova imagem, formada pelos pixels da imagem contidos dentro da *bounding box*. Essa nova imagem tem, portanto, as mesmas dimensões da *bounding box*.

A figura 18 exemplifica o processo de extração da placa com base na *bounding box* inferida pelo modelo de detecção de objetos.



Figura 18 – Extração da placa com base na *bounding box*

4.3.2.3 Transformação de perspectiva

Após extrair a imagem da placa, o próximo passo é utilizar a transformação de perspectiva, de forma que esta fique sem rotação e sem perspectiva. Essa operação tem grande potencial de padronização das imagens de placa, anulando variações que possam ocorrer referentes à posição do carro.

Essa transformação é feita com base nas coordenadas dos quatro cantos da placa, que definem o plano inicial, assim como as coordenadas dos quatro cantos na imagem final.

Para realizar isso é necessário primeiro realizar uma transformação nas coordenadas dos cantos para que o ponto de referência deixe de ser a origem da imagem inicial (do carro inteiro) e passe a ser a origem da imagem da placa.

Para isso, basta aplicar as equações:

$$x_2 = x_1 - x_{min} \quad (19)$$

$$y_2 = y_1 - y_{min} \quad (20)$$

Onde (x_1, y_1) são as coordenadas de um ponto referente à imagem do carro e (x_2, y_2) são as coordenadas referentes à imagem recortada da placa. (x_{min}, y_{min}) são os limite inferior de x e y na *bounding box*.

Por fim, basta definir as dimensões da imagem após a transformação. Essa definição é relativamente arbitrária, porém é ideal que a imagem final possua um tamanho grande o bastante para que não haja queda na resolução e que as proporções da largura e altura não sejam muito diferentes das proporções de uma placa de carro padrão, para que não ocorra muita distorção na imagem. Foi proposta então as dimensões de 500×200 px.

A figura 19 exemplifica o que acontece com a imagem da placa após a transformação. Além de a placa se ajustar muito melhor na imagem, suas dimensões ficam padronizadas em 500×200 px.

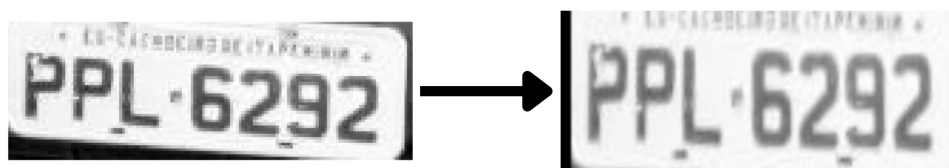


Figura 19 – Transformação de perspectiva com base nos quatro cantos da placa

4.3.2.4 Remoção das bordas

Com as dimensões da imagem padronizadas e a perspectiva corrigida, é possível, com base em conhecimento empírico acerca das características dimensionais da placa, reduzir ainda mais a imagem ao remover bordas desnecessárias, mantendo, na medida do possível, apenas os caracteres.

Essa operação é diferente em placas do modelo Brasil e Mercosul, pois estes modelos são diferentes nas dimensões das bordas. Nas placas do modelo Brasil são removidos 40 pixels do topo e 10 pixels da parte de baixo, além de 10 pixels na esquerda e na direita. Já nas placas do modelo Mercosul são removidos 40 pixels do topo, da esquerda e da direita, além de 10 pixels da parte de baixo. As figuras 20 e 21 mostram as medidas removidas das placas.

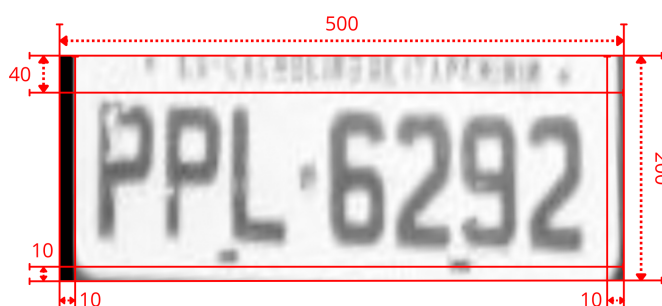


Figura 20 – Medidas para remoção de bordas das placas do padrão antigo

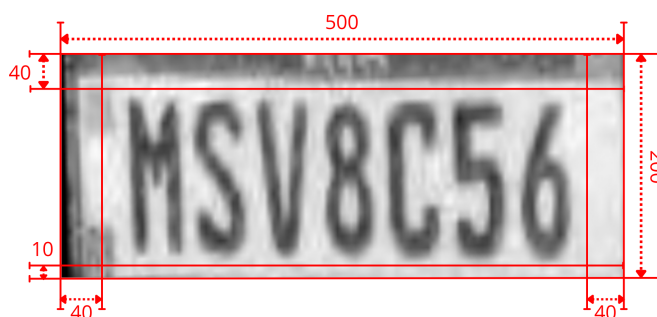


Figura 21 – Medidas para remoção de bordas das placas do padrão Mercosul

A figura 22 demonstra a transformação de remoção de bordas nas duas modalidades de placas. Essa operação ajuda a destacar a parte da imagem que realmente interessa à análise e remover o restante.

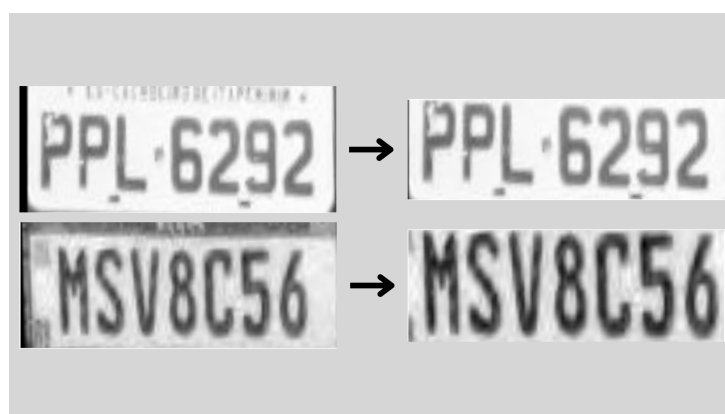


Figura 22 – Remoção de bordas das placas

4.3.2.5 Binarização

Com as bordas removidas, é então aplicada binarização na imagem da placa, para obter-se uma versão simplificada na qual os caracteres possuem destaque maior.

Para isso, é aplicada uma técnica de limiarização adaptativa que classifica cada pixel da imagem em escala de cinza como 0 ou 1, a depender do valor do pixel e dos pixels nos arredores. O algoritmo utilizado é o método de Sauvola com os parâmetros $k = 0.005$ e o tamanho de janela de 201 px. A utilização desse método evita que placas que não estão iluminadas de forma igual em toda a sua extensão tenham a qualidade da binarização comprometida.



Figura 23 – Imagem de placa binarizada com o método de Sauvola

4.3.2.6 Separação dos caracteres

A separação dos caracteres da imagem da placa binarizada é realizada em duas etapas: Primeiro é feita uma separação estimada, usando as dimensões conhecidas da placa como base, depois a separação é refinada com base nas características da imagem binarizada.

A separação inicial é simples. Sabe-se que cada placa possui 7 caracteres distribuídos de forma uniforme, portanto basta dividir a placa em 7 segmentos de igual largura para se obter uma boa estimativa das divisões dos caracteres, ao menos nas placas do modelo Mercosul. Nas placas do modelo antigo a distribuição não é uniforme por conta do hífen que separa as letras dos números. Para resolver isso, as placas desse modelo primeiro são divididas em duas: uma composta pelas 200 primeiras colunas de pixels da imagem e outra composta pelas 290 últimas colunas. Depois disso, a primeira parte é dividida em três partes iguais e a segunda parte em quatro partes iguais.

Após essa divisão inicial, a divisão é refinada. Como em tese, após a binarização, todo pixel que pertence a um caractere possui valor 1 e os demais pixels possuem valor 0, é feita uma varredura nos arredores do ponto de corte estimado procurando pela coluna com menos pixels com valor 1. No cenário ideal, será encontrado um ponto de corte em que todos os pixels possuem valor 0.

Por fim, a imagem é dividida em 7 pedaços com base nos melhores pontos de corte encontrados. Para padronizar as dimensões, os pedaços são expandidos horizontalmente com pixels de valor 0 até que a largura e altura possuam o mesmo valor.

A figura 24 mostra as etapas e o resultado da separação de caracteres para os dois modelos de placa.

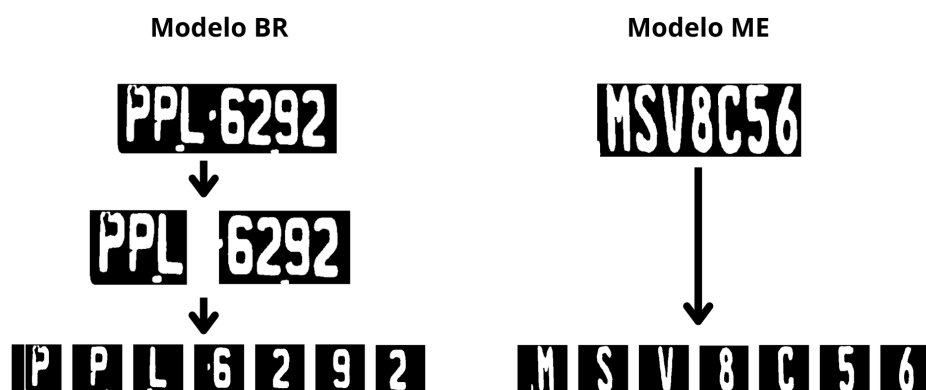


Figura 24 – Separação de caracteres para os dois modelos de placa

4.3.2.7 Limpeza das imagens de caracteres

A última etapa deste bloco é a limpeza das imagens de caracteres extraídas. Muitas dessas imagens acabam sendo geradas com ruído, que não tem nenhuma utilidade e pode atrapalhar a classificação dos caracteres. Esse ruído são pixels que não pertencem ao caractere em questão mas acabam sendo classificados como 1 na binarização.

Quando esses pixels estão desconectados do caractere é possível identificá-los com base na continuidade. Esse processo envolve basicamente classificar cada pixel de valor 1 da imagem como pertencente à um objeto, e em seguida eliminar todos os objetos com exceção daquele com maior área, que deve ser o caractere.

Após essa eliminação as imagens passam a conter consideravelmente menos ruído, como pode ser observado na figura 25.

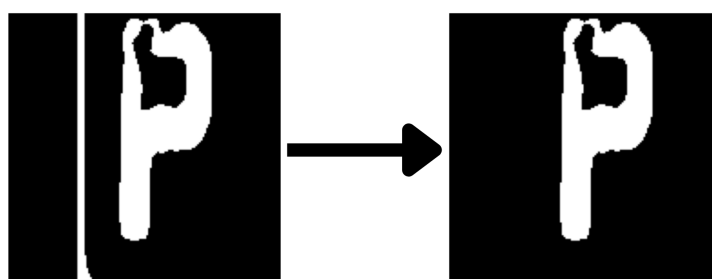


Figura 25 – Exemplo de limpeza de imagem de caractere com base em continuidade de objetos

4.3.3 Modelos de classificação de caracteres

Os modelos de classificação de caracteres têm a atribuição de receber uma imagem contendo um único caractere e classificá-lo entre as classes possíveis para aquele modelo. Como já é previamente conhecido, com base na posição dos caracteres, quando um caractere é uma letra ou um número, é possível criar modelos separados para classificação de letras e de números. Além disso, como também é sabido qual o modelo da placa, já que isso é inferido pelo modelo de detecção de objetos, é possível também que cada modelo seja especializado em caracteres do modelo Brasil ou do modelo Mercosul.

As figuras 26 e 27 mostram os diagramas das redes neurais utilizadas na classificação de letras e na classificação de números. A única diferença entre as duas é que a saída da rede de números é um vetor de 10 elementos, representando os 10 algarismos existentes, enquanto a rede de letras é um vetor de 26 valores, representando cada letra do alfabeto.

Os tipos de camadas utilizadas na rede neural seguindo a notação das figuras 26 e 27 foram:

- **Conv:** Camada convolucional
- **Gemm:** Camada linear *fully connected*

Além disso foram utilizadas as operações:

- **Relu:** Rectified Linear Unit
- **MaxPool:** Max pooling
- **Reshape:** Modificação da forma do tensor

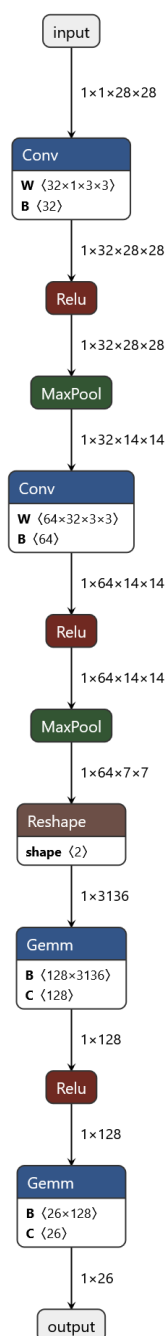


Figura 26 – Diagrama da rede neural de classificação de letras

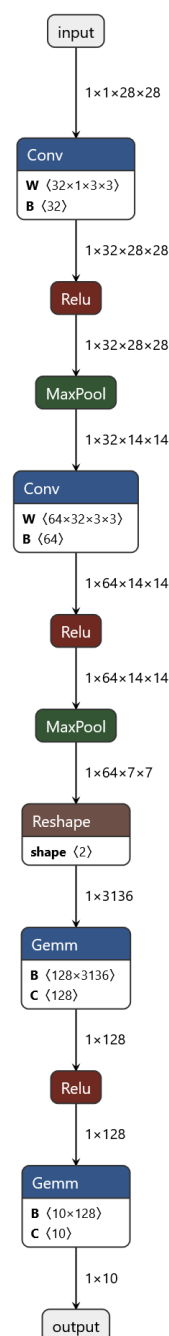


Figura 27 – Diagrama da rede neural de classificação de números

4.3.4 Modelos de extração e classificação de caracteres

Os modelos de extração e classificação de caracteres podem substituir o algoritmo de extração de caracteres e modelos de classificação de caracteres ao juntar ambas as tarefas em uma única solução integrada. Esses modelos devem extrair e classificar os caracteres diretamente da placa cortada da imagem original após a detecção pelo YOLO. A única transformação a ser feita na placa antes de alimentar para esse modelo é a transformação de perspectiva, pois esta garante uma padronização

muito maior das placas enviadas ao modelo.

A saída desses modelos é um conjunto de 7 vetores, cada um representando um dos caracteres da placa. Cada vetor é composto por 10 ou 26 valores, a depender se aquele vetor representa um número ou uma letra, o que depende da posição do caractere que aquele vetor representa.

Como as placas dos modelos Brasil e Mercosul possuem configurações diferentes, devem ser feitos dois modelos, um para cada tipo de placa. Esses modelos possuem uma arquitetura consideravelmente mais complexa que os modelos de classificação de caracteres isolados, podendo ser dividida em três blocos principais:

- **Backbone:**

O primeiro bloco da arquitetura é composto por múltiplas camadas convolucionais, funções de ativação ReLU, max poolings e conexões residuais. A função desse bloco é extrair *features* da imagem que serão utilizadas na classificação.

- **Average Pooling adaptativo:**

Esse bloco é a peça central da arquitetura, que divide a parte de extração de *features* da parte de classificação efetiva. A função desse bloco é nada mais que a separação dos caracteres, pois ele garante que sua saída serão 7 vetores, cada um contendo todas as *features* extraídas de um dos caracteres.

- **Cabeçalho:** Por fim, o último bloco é onde ocorre a classificação dos caracteres. Os 7 vetores que saem do *average pooling* adaptativo são divididos e cada um segue um caminho diferente, onde passam por uma camada linear *fully connected* e ativação ReLU, depois mais uma camada linear, cada vetor terminando com 10 ou 26 valores.

4.4 CONVERSÃO PARA EXECUÇÃO NA PLACA ROCK 3A

Para execução do modelo YOLO e dos modelos de classificação de caracteres na NPU placa Rock 3A, segundo a documentação da mesma, é necessário que esses modelos sejam convertidos para um formato compatível. Esse formato é o RKNN, desenvolvido para chips da Rockchip, como o utilizado na placa Rock 3A.

A Rockchip disponibiliza um software que permite a conversão de modelos no formato ONNX para RKNN. Ele possibilita também que o modelo seja quantizado para *uint8* durante a conversão. Para isso é necessário criar *datasets* de quantização, que serão utilizados pelos algoritmos de quantização e garantirão que o modelo quantizado se mantenha dentro de certos parâmetros.

O *dataset* de quantização não envolve rótulos, apenas imagens que podem ser utilizadas como entrada no modelo. No caso do YOLO, os dados de quantização devem ser parte das imagens de carros do *dataset* de treino desse modelo.

4.5 METODOLOGIA DE TESTE

Os testes para verificar a eficácia do algoritmo obtido se basearam em executar o programa em um conjunto de dados de teste, que foi separado no início do desenvolvimento da solução. Os dados de teste não foram utilizados em nenhuma etapa do desenvolvimento da solução além da própria etapa de teste, para não gerar viés nos resultados. As medições feitas nos testes foram:

- Percentual de imagens do conjunto de teste inferidas corretamente
- Tempo de detecção das placas com o modelo YOLO
- Tempo de processamento da imagem da placa (com ou sem extração de caracteres)
- Tempo de classificação dos caracteres

Os testes foram realizados tanto em um PC quanto na placa Rock 3A, e foram testados os dois modelos de algoritmo desenvolvidos.

5 DESENVOLVIMENTO E ANÁLISE DOS RESULTADOS OBTIDOS

Neste capítulo será discutido o que foi efetivamente feito para realização da solução proposta no capítulo 4, assim como a concretização da metodologia utilizada. Também são apresentadas as tecnologias utilizadas e problemas encontrados.

As etapas de desenvolvimento foram:

1. Preparação dos dados
2. Treinamento do modelo de detecção de placas
3. Implementação do algoritmo de extração de caracteres
4. Criação dos *datasets* de caracteres e de placas
5. Treinamento dos modelos de classificação de caracteres
6. Quantização e transformação dos modelos para formato compatível com o *hardware*
7. Preparação do *hardware*
8. Realização dos testes

5.1 PREPARAÇÃO DOS DADOS

O primeiro passo da preparação dos dados foi a separação em conjuntos de treino, validação e teste, cada qual com um propósito diferente:

- **Treino:** O conjunto de treino é utilizado para ajustar os modelos, que irão se adaptar para minimizar o erro de inferência nesses dados.
- **Validação:** Este conjunto é utilizado para, durante o treinamento dos modelos, garantir que o ajuste feito com os dados de treino se generalizam para dados não utilizados no treinamento.
- **Teste:** Estes dados não são utilizados em nenhum momento durante o desenvolvimento e ajustes dos modelos. Sua função é serem utilizados nos testes finais para verificar a eficácia do modelo desenvolvido.

O *dataset* original continha 10 mil imagens de carros, sendo que destas 5 mil eram com placas do tipo Brasil e 5 mil do tipo Mercosul. Os dados foram então divididos da seguinte forma: 70% para o conjunto de treino, 20% para o conjunto de validação e 10% para o conjunto de teste. Os dados colocados em cada conjunto foram selecionados aleatoriamente, mas garantindo que cada conjunto teria 50% de imagens com placas Mercosul e 50% com placas Brasil.

5.1.1 Organização dos dados

O treinamento do modelo de detecção de placas foi realizado com a biblioteca Ultralytics do python. Essa biblioteca, além de oferecer diversos modelos da família YOLO, já inclui o algoritmo de treinamento, necessitando apenas que o usuário deixe os dados organizados em um formato específico. Esse formato inclui deixar os dados de treino e validação em pastas separadas, além de escrever os arquivos de anotações em um formato específico.

Os arquivos de anotações são os arquivos que contêm as informações das *bounding boxes* e pontos-chave das imagens. O Ultralytics exige que eles sejam arquivos do tipo txt com o mesmo nome do arquivo de imagem ao qual se referem. Por exemplo, a imagem *img_000001.jpg* deve ter um arquivo *img_000001.txt* correspondente que contém as anotações daquela imagem.

O arquivo de anotações pode ter múltiplas linhas, cada um correspondendo a um objeto na imagem. Cada linha é composta da seguinte forma:

```
<classe> <x centro> <y centro> <largura> <altura> <x ponto 1>  
<y ponto 1> <visibilidade ponto 1> ... <x ponto n> <y ponto n>  
<visibilidade ponto n>
```

A classe é o número da classe daquele objeto. No caso do projeto existem duas classes: placa Brasil (0) e placa Mercosul (1). O *x centro* e *y centro* correspondem às coordenadas (*x*, *y*) do centro do objeto. A largura e altura, como o nome sugere, são a largura e altura do objeto. Por fim, cada ponto-chave é composto por três informações: coordenada *x*, coordenada *y* e visibilidade. Como em todas as imagens a placa está com os quatro cantos visíveis, a visibilidade é sempre 2, que é o valor que representa pontos visíveis. Todas as coordenadas, assim como altura e largura são normalizadas para que fiquem entre 0 e 1.

5.2 TREINAMENTO DO MODELO DE DETECÇÃO DE PLACAS

Com os dados preparados nas pastas, basta criar um arquivo de configuração no format YAML e escrever um simples script em python para iniciar o treinamento do modelo desejado. O arquivo de configuração deve conter as seguintes especificações:

- O caminho para a pasta que contém os dados
- O sub caminho para as pastas que contém os dados de treino e validação
- O número de classes
- Os nomes das classes

- O formato dos pontos-chave
- Índice espelhado dos pontos-chave

No caso deste projeto o número de classes é 2, cujos nomes foram definidos como *placa_br* e *placa_me*. O formato dos pontos-chave deve ser uma tupla contendo dois valores: O número de pontos-chave por objeto e a dimensão dos pontos-chave, que é 3 (largura, altura, visibilidade). O índice espelhado dos pontos-chave é uma lista de índices de 0 a n sendo que n é a quantidade de pontos-chave por objeto. A ordenação dos índices deve indicar como os índices devem ser tratados caso a imagem seja espelhada horizontalmente. Isso é necessário pois o Ultralytics faz isso com algumas imagens durante o processo de treinamento.

Com o arquivo de configuração preparado é possível executar o script de treinamento, que é bastante simples. Precisa primeiro importar a biblioteca Ultralytics e criar um objeto da classe YOLO, que recebe como argumento o nome do modelo YOLO desejado, que no caso é *yolo26n-pose*. Em seguida basta chamar o método *train*, indicando o caminho para o arquivo de configuração que foi criado, o tamanho do *batch* desejado, o número de épocas e o dispositivo no qual executar o modelo.

Foram treinadas 100 épocas no dispositivo *cuda* (GPU do computador). O tamanho do *batch* foi deixado como -1, que faz com que o próprio Ultralytics defina o melhor tamanho de *batch* com base na capacidade da GPU. As 100 épocas se mostraram suficientes para ajustar o modelo, que ao final do treinamento estava com um erro bem pequeno e estável, tanto na detecção dos objetos como na classificação e detecção dos pontos-chave.

A figura 28 mostra três curvas do erro do modelo ao longo do treinamento. A primeira é o erro do ajuste da *bounding box*, a segunda é o erro dos pontos-chave e a terceira é o erro de classificação.

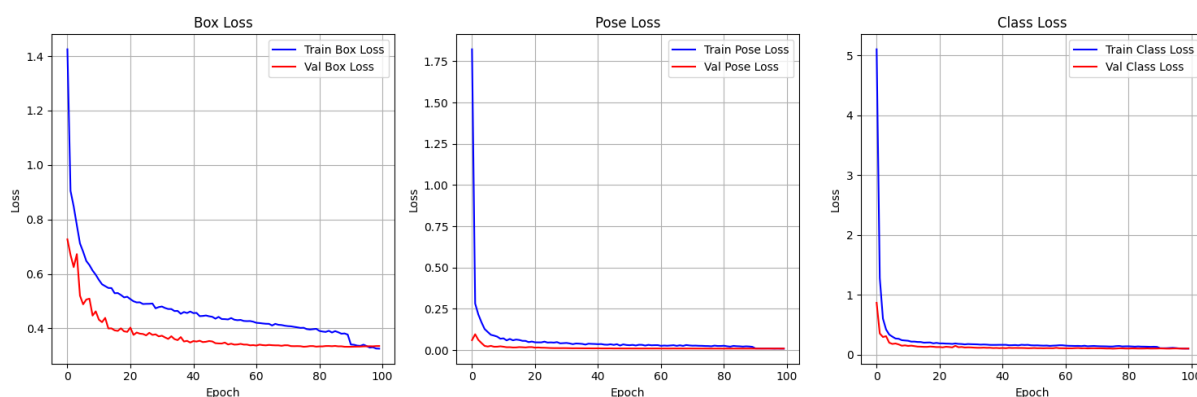


Figura 28 – Curvas de treinamento do modelo de detecção de placas

5.2.1 Exportação do modelo YOLO

Findado o treinamento o Ultralytics automaticamente exporta um arquivo com os pesos do modelo ajustados. Porém esse formato é compatível apenas para realizar inferência com a própria biblioteca Ultralytics. Para obter o modelo em um formato com mais portabilidade, basta criar um novo script, carregar o modelo treinado e utilizar o método *export* para obter o modelo em um arquivo do tipo ONNX. Esse formato permite inferência em diversas plataformas através da biblioteca *onnx runtime*, mas também permite que o modelo seja carregado pelo software que realiza a conversão para o formato RKNN, compatível com a NPU da placa Rock 3A.

5.3 IMPLEMENTAÇÃO DO ALGORITMO DE EXTRAÇÃO DE CARACTERES

Após o ajuste do modelo de detecção de placas, foi criado o script de processamento das imagens e extração dos caracteres. Esse script foi feito em python, e utilizou a biblioteca *onnx runtime* para realizar inferência com o modelo de detecção de placas exportado para formato ONNX.

As etapas do script são as seguintes:

1. Inferência com o modelo de detecção de placas
2. Criação de uma nova imagem contendo apenas a placa, cortada da imagem original com base na *bounding box* gerada pelo modelo de detecção
3. Correção de perspectiva
4. Remoção das bordas da placa com base no tipo de placa verificado pelo modelo de detecção
5. Binarização da imagem
6. Separação dos caracteres
7. Limpeza das imagens dos caracteres

A seguir será discutida em mais detalhes a implementação de cada etapa do script.

5.3.1 Inferência com o modelo de detecção de placas

O modelo YOLO foi treinado com a biblioteca Ultralytics e exportado para formato ONNX, podendo agora ser utilizado para inferência com a biblioteca do python *onnx runtime*. A implementação da inferência, então, incluiu carregar o arquivo do modelo e criar uma função que recebe a imagem e devolve a inferência.

A função não se limita simplesmente a alimentar a imagem ao modelo, pois é necessário algum pré-processamento da imagem antes disso. Primeiramente a imagem deve ser redimensionada para o formato 640×640 px, que é a dimensão utilizada pelo YOLO. Em seguida os valores dos pixels devem ser normalizados para que fiquem entre 0 e 1.

O redimensionamento foi feito com a biblioteca de manipulação de imagens OpenCV, assim como o próprio carregamento da imagem. O OpenCV carrega a imagem como um tensor de dimensão (H,W,C), onde H é a altura da imagem, W é a largura e C é o número de canais, que é 3 para imagens coloridas e 1 para imagens em escala de cinza. Cada valor representa a intensidade de um pixel em um dado canal (para imagens coloridas os canais são vermelho, verde, azul), que é representada por um número inteiro de 0 a 255 (8 *bits*).

O OpenCV oferece uma função para redimensionamento de imagem, já a normalização pode ser feito simplesmente dividindo o tensor por 255. Com isso, a imagem fica pronta para ser alimentada ao modelo.

A última etapa dessa implementação é a interpretação da saída do modelo. Ao alimentar o modelo com a imagem, ele devolve uma saída no formato de uma matriz 300×18 . Cada linha dessa matriz representa uma detecção representada por 18 valores. A única detecção que nos interessa é a primeira, pois as detecções são ordenadas em ordem de confiança. O modelo é obrigado a gerar 300 detecções pela forma como foi construído, mas a maioria delas não representa nenhum objeto. Como nosso objetivo é analisar apenas uma placa por imagem, pegamos apenas a primeira detecção e descartamos as demais, de forma que se houver mais de uma placa, apenas a mais visível será analisada.

Dos 18 valores que representam a detecção a interpretação é a seguinte:

- Os 4 primeiros valores representam a *bounding box* do objeto na seguinte ordem:
 x_1, y_1, x_2, y_2
- O quinto valor representa a confiança da detecção, que vai de 0 a 1
- A classe do objeto (*placa_br* ou *placa_me*) é representada pelo sexto valor
- As coordenadas dos pontos-chave são os valores seguintes

Como a imagem foi redimensionada para 640×640 é necessário converter as coordenadas dos pontos-chave e da *bounding box* para a dimensão da imagem original. Para isso basta multiplicar as coordenadas *x* pela largura da imagem original e as coordenadas *y* pela altura e dividir todas por 640. Assim temos as coordenadas nas dimensões da imagem original.

5.3.2 Criação de nova imagem contendo apenas a placa

Com base na *bounding box* gerada pelo modelo, é possível cortar a imagem original e gerar uma imagem que contenha apenas a placa detectada. Antes disso, porém, convertemos a imagem para escala de cinza, de forma que seja representada por uma única matriz ($H \times W$). A conversão para escala de cinza é feita pela biblioteca OpenCV.

Depois disso, basta criar uma nova matriz apenas com os pixels cujas coordenadas (x, y) obedecem às limitações: $x_1 \leq x \leq x_2$ e $y_1 \leq y \leq y_2$, sendo que x_1, x_2, y_1, y_2 são os limites da *bounding box* na imagem original.

5.3.3 Correção de perspectiva

A correção de perspectiva é realizada pela biblioteca OpenCV em uma sequência de etapas:

1. Definição das coordenadas dos pontos antes da transformação
2. Definição das coordenadas dos pontos depois da transformação
3. Geração da matriz de transformação
4. Transformação com base na imagem e na matriz

Os pontos antes da transformação nada mais são do que as coordenadas dos pontos-chave encontrados pelo YOLO. A única questão é que eles precisam ser transformados para que a referência das coordenadas seja a imagem cortada, não a imagem original. A operação para essa transformação foi discutida no capítulo 4.

Os pontos pós transformação nada mais são que os quatro cantos de uma imagem hipotética de dimensões $W_2 \times H_2$. Essa dimensão será a dimensão da imagem após a transformação, que foi definida como 500×200 . Portanto os 4 pontos pós transformação são: $(0, 0)$, $(499, 0)$, $(499, 199)$, $(0, 199)$.

Definidos os pontos pode ser gerada a matriz de transformação com base neles, o que é feito através de uma função do OpenCV. Em seguida, basta utilizar uma outra função do OpeCV para obter a imagem transformada com base na matriz de transformação, na imagem inicial (imagem cortada da placa) e nas dimensões desejadas da imagem final.

Com isso, temos uma função que recebe uma imagem com dimensões $(w \times h)$ de uma placa e o *output* do modelo YOLO e devolve uma imagem de dimensões (500×200) com a perspectiva da placa corrigida.

5.3.4 Remoção das bordas das placas e binarização

A remoção das bordas das placas implica basicamente em cortar as imagens de acordo com as dimensões do tipo de placa. Para fazer isso basta limitar os pixels que devem fazer parte da imagem cortada. Considerando as coordenadas (x, y) dos pixels basta seguir o padrão:

$$c_l \leq x \leq 500 - c_r \quad (21)$$

$$c_t \leq y \leq 200 - c_b \quad (22)$$

Sendo que c_l , c_r , c_t e c_b são a quantidade de pixels a cortar na esquerda, direita, topo e parte de baixo da imagem, respectivamente. Esses valores são diferentes para placas do tipo Brasil e Mercosul, como discutido no capítulo 4.

A binarização, que utilizou o método de Sauvola, foi feita utilizando algumas funções diferentes do OpenCV. O primeiro passo foi gerar uma imagem na qual cada pixel era a média dos pixels vizinhos na imagem da placa. Para isso, foi utilizada uma função do OpenCV que faz exatamente essa tarefa a partir de uma imagem e um tamanho de janela, que para o nosso caso foi escolhido 201 pixels. Esse é um tamanho relativamente grande considerando que a imagem possui 500 pixels de largura, mas o propósito é justamente não permitir flutuação demais para que o resultado seja menos afetado por ruído e capture apenas a variação de luminosidade ao longo da imagem.

Em seguida foi calculado a mesma coisa para o quadrado da imagem (a intensidade de cada pixel foi elevada ao quadrado). Por fim, utilizando simples operações matemáticas, foi calculado o limiar para cada pixel da imagem, e foi gerada uma imagem binarizada na qual cada pixel era 0 ou 1, a depender se o pixel da imagem original naquela posição era menor ou maior que o pixel na matriz de limiar para aquela posição.

As equações para cálculo da matriz de limiar foram discutidas mais a fundo no capítulo 2.

5.3.5 Separação dos caracteres

A implementação da separação de caracteres da imagem binarizada se baseou na seguinte sequência de eventos:

1. É gerada uma estimativa dos pontos de separação, cada uma representada como uma coordenada x da imagem da placa
2. A função itera por cada um desses pontos, tentando encontrar um ponto de divisão melhor nos seus arredores
3. Os pontos de divisão refinados são utilizados para dividir a imagem em n pedaços, cada um contendo apenas um caractere

4. Cada imagem de caractere é "estampada" em uma imagem preta quadrada

Para funcionar a função precisa dos seguintes parâmetros:

- A imagem a ser separada
- A quantidade de caracteres presentes naquela imagem
- O tamanho da janela na qual um ponto de corte melhor será procurado (padronizado em 40)

A quantidade de caracteres presentes na imagem define o número de separações a ser feito. Isso é importante pois enquanto as placas do tipo Mercosul são separadas de uma só vez, as placas do tipo brasil são primeiro divididas em duas para que depois cada parte seja separada, como já discutido no capítulo 4.

O tamanho da janela define a até quantos pixels de distância do ponto de separação inicialmente estimado será buscado um ponto de separação melhor. A avaliação dos candidatos a pontos de separação é simples: Cada ponto é uma coluna da imagem, sendo então somadas a intensidade de todos os pixels naquela coluna. Como a binarização já foi feita, pixels que não pertencem a nenhum caractere possuem valor 0, enquanto os que pertencem possuem valor 1. Portanto, a coluna cuja soma das intensidades dos pixels for menor (idealmente 0) é melhor para a divisão, pois provavelmente é um espaço vazio entre dois caracteres.

Depois de dividir a imagem, porém, cada uma terá uma largura diferente. Isso é então padronizado ao gerar uma imagem quadrada preta $H \times H$ (onde H é a altura da imagem da placa) e "estampando" a imagem com o caractere no meio dessa imagem. Isso garante que todas as imagens de caracteres terão as mesmas dimensões.

5.3.6 Limpeza das imagens de caracteres

Para a limpeza das imagens de caracteres foi utilizada a biblioteca *scikit-image* do Python. Ela oferece uma função que, ao receber uma imagem binarizada, como são as imagens de caracteres, mapeia cada pixel branco (valor 1) como parte de um objeto, com base na continuidade. Todo pixel branco que está adjacente a um outro pixel branco faz parte do mesmo objeto que este. Dessa forma a imagem é dividida em vários objetos, e representada por uma matriz na qual cada pixel é um "rótulo", um valor inteiro que indica a qual objeto pertence. Pixels com valor zero são pixels pretos, portanto não pertencem a objeto nenhum.

O *scikit-image* então oferece uma outra função na qual recebe essa matriz de rótulos e calcula características dos objetos, como altura, largura, área, etc. A única que nos interessa é a área, pois ela é o melhor indicativo de qual é o maior objeto da imagem, que seria o caractere. Com base nisso, então, pixels que não pertencem ao

objeto de maior área são eliminados da imagem do caractere, passando a ter valor zero. Isso elimina ruído e deixa a imagem do caractere mais limpa, minimizando a propensão à erros de classificação daquele caractere.

5.4 CRIAÇÃO DOS *DATASETS* DE CARACTERES E DE PLACAS

Finalizados todos os algoritmos de processamento das imagens passa a ser possível criar os *datasets* para o treinamento dos modelos de classificação. Como temos duas abordagens distintas de modelos, foram criados dois *datasets*: Um com imagens de caracteres binarizadas e limpas e uma com imagens em escala de cinza de placas com perspectiva corrigida.

Para criar o *dataset* de caracteres todas as imagens do conjunto de treino e validação inicial fora submetidas ao *pipeline* completo desenvolvido até aqui. Passaram pelo modelo YOLO, onde a placa foi detectada, foram cortadas para manter só a imagem da placa, convertidas para escala de cinza, tiveram sua perspectiva corrigida, passaram pela remoção de bordas, binarização, separação de caracteres e limpeza. Ao final de tudo, cada imagem de caractere foi rotulada com o caractere contido naquela imagem, que era sabido pelo fato de termos nos dados originais as informações das placas dos carros contidos neles. Todas as imagens foram então salvas em pastas de acordo com:

- O tipo de caractere (número ou letra)
- O tipo de placa (Brasil ou Mercosul)

Os nomes das imagens salvas foram compostos pelo caractere representado na imagem, o nome da imagem original da qual o caractere foi extraído e a posição do caractere na placa (0 - 6).

O *dataset* de placas teve um processo de criação mais simples. As imagens originais passaram pela detecção, corte, transformação para escala de cinza e correção de perspectiva, apenas. Com isso já se tinham imagens de placas com dimensões padronizadas, que foram então salvas em pastas separadas apenas pelo tipo de placa (Brasil, Mercosul). Os nomes das imagens continham os caracteres da placa e mais um código identificador.

5.5 TREINAMENTO DOS MODELOS DE CLASSIFICAÇÃO DE CARACTERES

Foram feitos dois treinamentos de modelos de classificação de caracteres. O primeiro de modelos de classificação de caracteres isolados, como os gerados pelo *pipeline* completo de extração de caracteres, e o outro de modelos de extração e classificação, com intenção de classificar os caracteres direto das imagens de placas

extraídas e com perspectiva corrigida. Ambos foram realizados utilizando a biblioteca PyTorch e ao final do treinamento exportados para o formato ONNX.

Os treinamentos foram realizados utilizando o seguinte ciclo:

1. *Batch* de imagens é alimentado ao modelo
2. É calculado o *Loss* do *Batch* (*Cross Entropy*)
3. É realizado *backpropagation* para cálculo do gradiente
4. Os pesos do modelo são atualizados

Esses passos são repetidos até que o modelo tenha sido alimentado com todos os dados do *dataset*. É então finalizada a época, o modelo é avaliado e tudo é feito novamente até que o número de épocas determinado inicialmente seja atingido.

5.5.1 Classificação de caracteres isolados

As características específicas do modelo de classificação de caracteres isolados são os seguintes parâmetros de treinamento:

- *Optimizer*: Adam com *learning rate* de 0.001
- Tamanho do *batch*: 32
- Número de épocas: 10

Foram treinados um total de quatro modelos distintos, cada um especializado em um tipo de caractere e um tipo de placa, seguindo a divisão feita na criação do *dataset* de caracteres.

As figuras 29, 30, 31 e 32 mostram as curvas do *Loss* de treinamento e validação, além da acurácia de validação de cada modelo ao longo das épocas de treinamento. Esses dados foram coletados ao fim de cada época com intenção de registrar a evolução do modelo ao longo do treinamento.

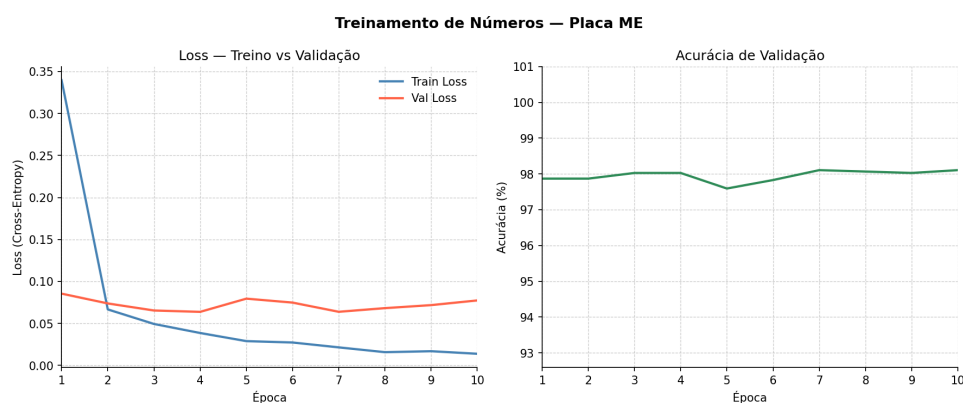


Figura 29 – Curvas de treinamento do modelo de classificação de números em placas do tipo Mercosul

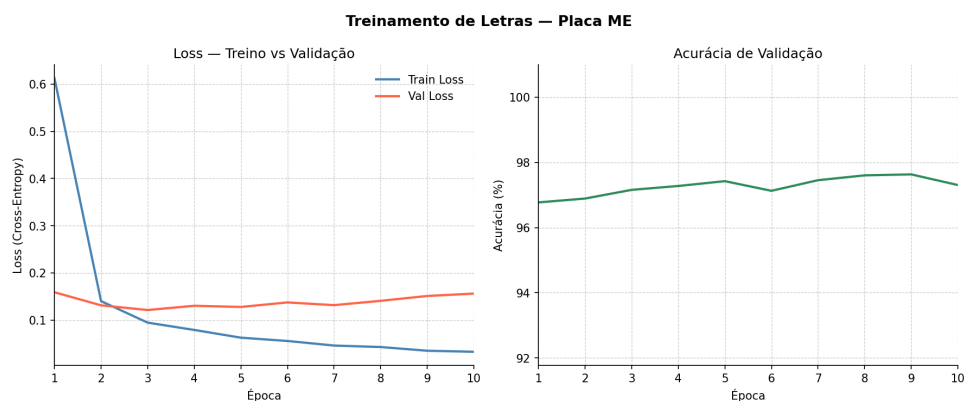


Figura 30 – Curvas de treinamento do modelo de classificação de letras em placas do tipo Mercosul

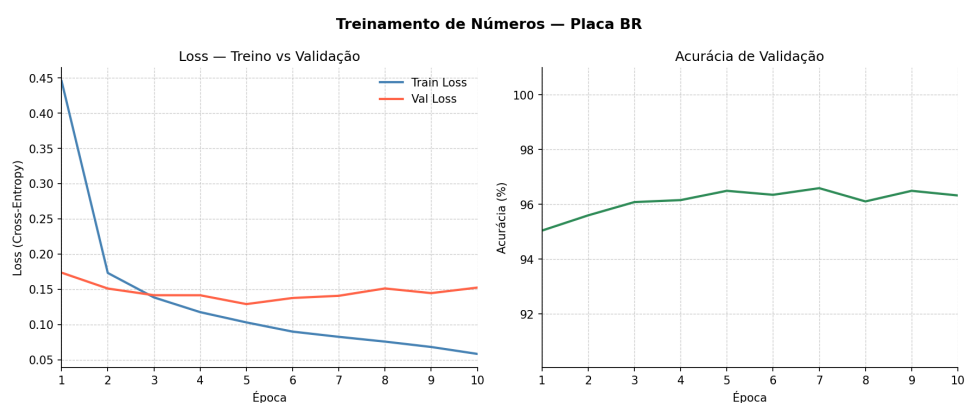


Figura 31 – Curvas de treinamento do modelo de classificação de números em placas do tipo Brasil

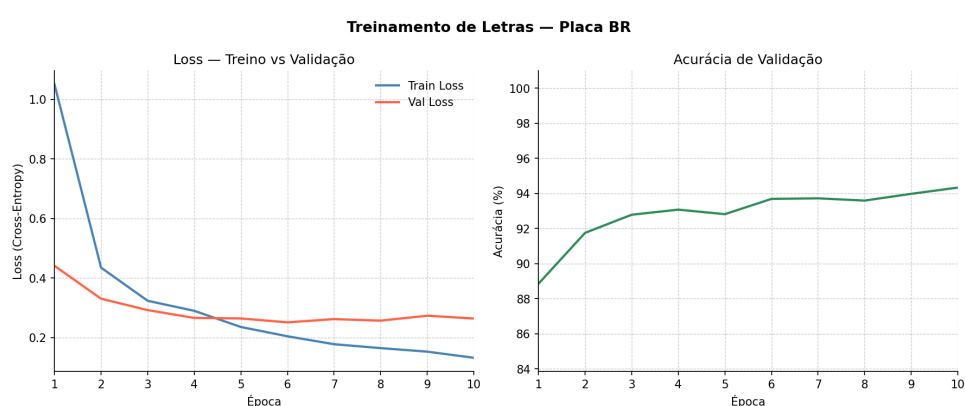


Figura 32 – Curvas de treinamento do modelo de classificação de letras em placas do tipo Brasil

5.5.2 Extração e classificação simultânea

O treinamento dos modelos de extração e classificação seguiu um processo parecido. Esse modelo, diferente dos modelos de classificação de caracteres isolados,

recebe uma imagem de placa e classifica todos os seus caracteres de uma só vez. O objetivo é que o modelo aprenda a identificar a posição dos caracteres e ao mesmo tempo classificá-los.

Esse modelo, por ser maior e mais complexo, utilizou também algumas técnicas mais refinadas de treinamento. Uma delas foi o *learning rate* variável, que aumenta ou diminui ao longo do treinamento. Outra foi o *early stopping*, que interrompe o treinamento ao identificar que a performance está estagnada.

Os parâmetros de treinamento foram:

- *Optimizer*: AdamW com *learning rate* de 0.003 e *weight decay* de 0.004
- Tamanho do *batch*: 64
- Número de épocas: 100
- Paciência: 20

A paciência determina a quantidade de épocas que podem ocorrer sem melhoras na acurácia do modelo. Após essa quantidade de épocas sem melhoria o treinamento é interrompido.

Foram treinados dois modelos distintos, um para placas do tipo Brasil e outro para placas do tipo Mercosul. As figuras 33 e 34 mostram a evolução da *Loss*, da acurácia por caractere e da acurácia por placa ao longo das épocas de treinamento.

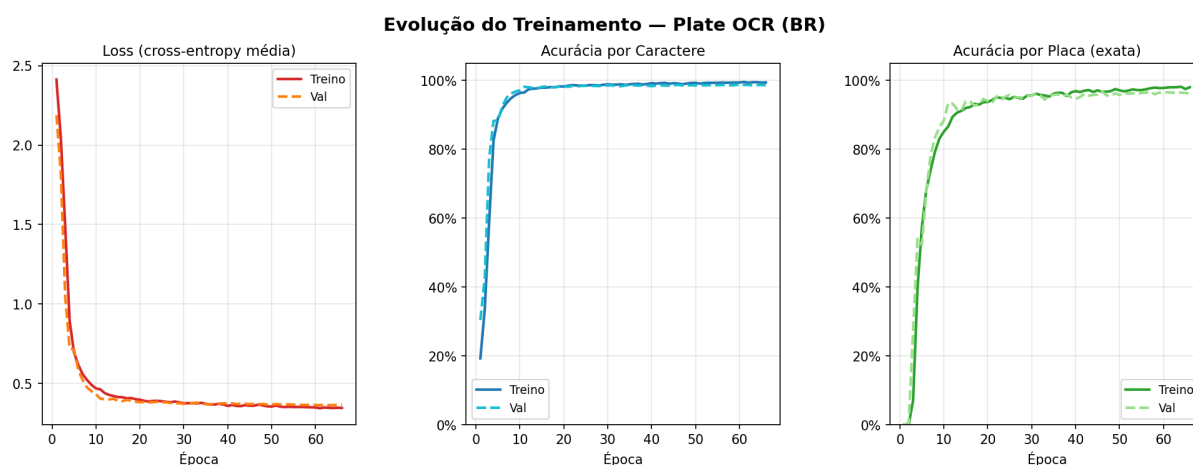


Figura 33 – Curvas de treinamento do modelo de classificação em placas do tipo Brasil

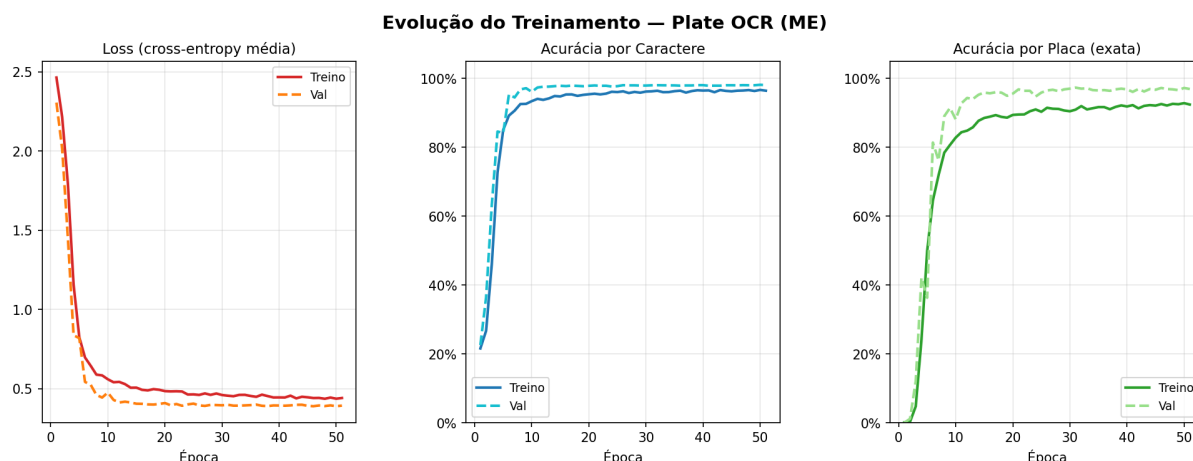


Figura 34 – Curvas de treinamento do modelo de classificação em placas do tipo Mercosul

5.6 QUANTIZAÇÃO E TRANSFORMAÇÃO DOS MODELOS PARA FORMATO COMPATÍVEL COM O *HARDWARE*

Todos os modelos treinados foram convertidos para o formato ONNX, que pode ser utilizado para execução na placa Rock 3A, porém sem tirar proveito da NPU. Para que a NPU possa ser utilizada, é necessário que os modelos sejam convertidos para o formato RKNN. A Rockchip, fabricante do chip de processamento utilizado na Rock 3A, disponibiliza um *software* na forma de uma biblioteca do python que permite a compilação dos modelos nesse formato. Porém, é necessário que o programa seja executado em um ambiente *linux* da distribuição Ubuntu versão 24.

É possível, no entanto, criar um ambiente com essas condições dentro do *windows* ou outros sistemas operacionais através de um *container* Docker. Um *container* Docker é um ambiente isolado dentro de um computador no qual pode ser instalado *software* sem afetar a máquina que contém o *container*. Esse *container* pode ter qualquer sistema operacional compatível com a máquina, incluindo o Ubuntu 24.

Foi então criado um *container* no qual foi instalado o python e todas as bibliotecas necessárias, incluindo o *rknn toolkit*, responsável pela conversão.

O código de conversão é um *script* simples, que necessita apenas que se aponte para o arquivo ONNX de origem, se preencha algumas informações de configuração e aponte para o *dataset* de quantização, caso se queira quantizar o modelo. A quantização tem potencial de aumentar consideravelmente a performance na NPU, sendo portanto altamente recomendada. As configurações que devem ser informadas são basicamente o modelo do chip, o nível de otimização, o algoritmo de quantização e os valores de normalização dos dados.

Como as imagens são carregadas como matrizes de números inteiros de 8 *bits*, podem ser necessárias transformações de normalização nas imagens. No modelo

YOLO, por exemplo, os pixels devem ser convertidos para valores reais de 0 a 1. Portanto, é necessário dividir os valores de 0 a 255 iniciais por 255. Isso deve ser informado para que a quantização funcione.

5.6.1 Resultados da conversão

O sucesso da conversão variou conforme o modelo:

- YOLO: sucesso na conversão porém sem quantização
- Modelo de caracteres isolados: sucesso na conversão e na quantização
- Modelo de extração e classificação: falha na conversão

Não foi possível identificar o que causou a falha na quantização do YOLO e na conversão dos modelos de extração e classificação, tendo sido tentadas várias abordagens. No caso do YOLO, a quantização é finalizada e o modelo é exportado, mas ao executar na placa os resultados saíam errados, já no caso do modelo de classificação e extração a conversão nem concluiu.

A conclusão que se chegou a respeito destes erros é que algo relativo à arquitetura dos modelos não é suportado, embora não se saiba exatamente o que.

Apesar dessas falhas, a execução na placa ainda é possível com todos os modelos. Apenas não sendo possível executar na NPU o modelo que não foi convertido, sendo necessário o uso da CPU. Já o YOLO pode ser executado na NPU normalmente, perdendo apenas os ganhos de performance que a quantização poderia proporcionar.

5.6.2 Criação dos datasets de quantização

Os *datasets* de quantização foram criados separando imagens do conjunto de treinamento de cada modelo, colocando em uma pasta separada e criando um arquivo de texto contendo o caminho de cada imagem. Para a quantização bastava apontar o caminho para este arquivo que o *rknn toolkit* se encarregava do resto do processo.

5.7 PREPARAÇÃO DO HARDWARE

A preparação da placa Rock 3A para execução do algoritmo envolveu alguns passos

5.8 TESTES REALIZADOS

6 CONCLUSÃO

Instruções da Coordenação do PFC:

A Conclusão deve apresentar:

- Uma **síntese** do problema tratado, da solução proposta e dos principais resultados obtidos;
- Uma discussão sobre o que foi atingido no PFC em relação aos objetivos inicialmente traçados e sobre as limitações encontradas;
- Uma discussão sobre possíveis aprimoramentos;
- Destacar os impactos do PFC para a empresa/clientes da empresa/instituto de pesquisa, além de possíveis impactos organizacionais, tecnológicos, financeiros, éticos, ecológicos, etc.
- Sugestão de trabalhos futuros (como se poderia dar continuidade ao PFC; aplicar o desenvolvimento realizado no PFC a outros problemas/processos; etc)

REFERÊNCIAS

CASEY, Richard G.; LECOLINET, Eric. A Survey of Methods and Strategies in Character Segmentation. **IEEE Transactions on Pattern Analysis and Machine Intelligence**, IEEE, v. 18, n. 7, p. 690–706, 1996. Revisão abrangente de técnicas de segmentação, incluindo histograma de projeção vertical.

CHAPMAN, Peter; CLINTON, Julian; KERBER, Randy; KHABAZA, Thomas; REINARTZ, Thomas; SHEARER, Colin; WIRTH, Rüdiger. **CRISP-DM 1.0: Step-by-step data mining guide**. [S.l.], 2000. Technical report. Disponível em: <https://www.crisp-dm.org>.

CHEN, Yunji; XIE, Yuan; SONG, Linghao; CHEN, Fan; TANG, Tianshi. A Survey of Accelerator Architectures for Deep Neural Networks. **Engineering**, Elsevier, v. 6, n. 3, p. 264–274, 2020. Panorama de aceleradores de DNN incluindo NPUs embarcadas como as encontradas em SoCs móveis e de borda.

FAYYAD, Usama M.; PIATETSKY-SHAPIRO, Gregory; SMYTH, Padhraic. From Data Mining to Knowledge Discovery in Databases. **AI Magazine**, v. 17, n. 3, p. 37–54, 1996. DOI: 10.1609/aimag.v17i3.1230. Disponível em: <https://ojs.aaai.org/index.php/aimagazine/article/view/1230>.

IBM INC. **What are convolutional neural networks?** [S.l.: s.n.], 2024. <https://www.ibm.com/>. Acessado em março de 2026. Disponível em: <https://www.ibm.com/think/topics/convolutional-neural-networks>.

JACOB, Benoit; KLIGYS, Skirmantas; CHEN, Bo; ZHU, Menglong; TANG, Matthew; HOWARD, Andrew; ADAM, Hartwig; KALENICHENKO, Dmitry. Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference, p. 2704–2713, 2018. Artigo seminal sobre quantização int8 com zero-point; base do esquema usado no TFLite e no RKNN toolkit.

KRIZHEVSKY, Alex; SUTSKEVER, Ilya; HINTON, Geoffrey E. ImageNet Classification with Deep Convolutional Neural Networks. In: **ADVANCES in Neural Information Processing Systems 25 (NIPS 2012)**. [S.l.: s.n.], 2012. p. 1097–1105. Disponível em: <https://papers.nips.cc/paper/4824-imagenet-classification-with-deep-convolutional-neural-networks>.

LAROCA, R.; CARDOSO, E. V.; LUCIO, D. R.; ESTEVAM, V.; MENOTTI, D. On the Cross-Dataset Generalization in License Plate Recognition. In: **INTERNATIONAL Conference on Computer Vision Theory and Applications (VISAPP)**. [S.l.: s.n.], fev. 2022. p. 166–178. DOI: 10.5220/0010846800003124.

LECUN, Yann; BOTTOU, Léon; BENGIO, Yoshua; HAFFNER, Patrick. Gradient-Based Learning Applied to Document Recognition. **Proceedings of the IEEE**, v. 86, n. 11, p. 2278–2324, 1998. DOI: 10.1109/5.726791. Disponível em: <https://doi.org/10.1109/5.726791>.

LIN, Tsung-Yi; MAIRE, Michael; BELONGIE, Serge; HAYS, James; PERONA, Pietro; RAMANAN, Deva; DOLLÁR, Piotr; ZITNICK, C. Lawrence. Microsoft COCO: Common Objects in Context. In: SPRINGER. EUROPEAN Conference on Computer Vision (ECCV). [S.l.: s.n.], 2014. p. 740–755.

REDMON, Joseph; DIVVALA, Santosh; GIRSHICK, Ross; FARHADI, Ali. You Only Look Once: Unified, Real-Time Object Detection. In: PROCEEDINGS of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR). [S.l.: s.n.], 2016. p. 779–788.

RUMELHART, David E.; HINTON, Geoffrey E.; WILLIAMS, Ronald J. Learning Representations by Back-propagating Errors. **Nature**, Nature Publishing Group, v. 323, p. 533–536, 1986. Artigo seminal que formalizou o algoritmo de backpropagation.

SAHA, Sumit. A Comprehensive Guide to Convolutional Neural Networks — the ELI5 way. **Medium**, 2018.

SAUVOLA, Jaakko; PIETIKÄINEN, Matti. Adaptive Document Image Binarization. **Pattern Recognition**, Elsevier, v. 33, n. 2, p. 225–236, 2000. Referência para binarização adaptativa local, análoga à abordagem por faixas do projeto.

THEAILEARNER. **Perspective Transformation**. [S.l.: s.n.], 2023.
<https://theailearner.com>. Acessado em março de 2026. Disponível em:
<https://theailearner.com/tag/cv2-getperspectivetransform/>.

ZOU, Zhengxia; SHI, Zhenwei; GUO, Yuhong; YE, Jieping. Object Detection in 20 Years: A Survey. **arXiv preprint arXiv:1905.05055**, 2019.

APÊNDICE A – DESCRIÇÃO 1

Textos elaborados pelo autor, a fim de completar a sua argumentação. Deve ser precedido da palavra APÊNDICE, identificada por letras maiúsculas consecutivas, travessão e pelo respectivo título. Utilizam-se letras maiúsculas dobradas quando esgotadas as letras do alfabeto.

ANEXO A – DESCRIÇÃO 2

São documentos não elaborados pelo autor que servem como fundamentação (mapas, leis, estatutos). Deve ser precedido da palavra ANEXO, identificada por letras maiúsculas consecutivas, travessão e pelo respectivo título. Utilizam-se letras maiúsculas dobradas quando esgotadas as letras do alfabeto.